

HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical
DiscoveryBonet Chaple

HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery

Ruperto Pedro Bonet Chaple

ruperto.bonet@modelphysmat.com

Independent Researcher, London, United Kingdom

Editor:

Note on data provenance and prior-draft corrections. Two claims from early drafts of this paper did not survive verification against the raw per-task result files and are not repeated in this version: a “73% runtime reduction” ($3.7\times$ speedup) and a subsequently reported $1.73\times/1.64\times$ speedup, both for HypatiaX over the neural-network baseline. The verified result is the opposite: across the five 74-task v3 seed files held in the repository (370 task runs audited; a separate PCA block of the same table, also 370 runs, shows the same direction), the hybrid system is *slower* than the pure neural-network baseline on every seed, by a factor of $3.8\times$ to $11.2\times$ on mean time, whether measured over all tasks or over LLM-routed tasks only (Table 10, §10.5). All seed-42 DeFi figures in this paper (the near-perfect-rate and decision-attribution-bug analysis of §10.4, Table 8, and Table 9) are computed from `hypatiaux_defi_benchmark_v3_results_seed42.json` as it currently exists in the repository (canonical location `hypatiaux/data/results/comparison_results/noise-noiseless/noiseless/defi`; identical copies under `hypatiaux-dev/data_exp` and `data_old`; md5 `cbf503d0b3985b4c1bccf7ed18e0316b`). This file’s raw figures — `hybrid.test_r2` > 0.99 on 67/74 tasks (90.5%), `pure_llm.success` on 45/74 (60.8%), and 5 pure-LLM catastrophic failures ($R^2 < -10$) — match this paper’s own headline raw numbers exactly, and are the basis for every corrected figure reported below and in §10.4: 44/74 (59.5%) corrected near-perfect rate, tied with pure LLM tier-by-tier, and 23 uncorroborated successes (3 easy / 11 medium / 9 hard). Two other v3-labelled seed-42 files exist in the repository (in `hypatiaux-dev/data/data_dev/data_last`, md5 `89037935469176741a5fb4479f2c725f`, and in `hypatiaux/data_old`) with slightly different task-level results (73/74 and 67/74 raw `hybrid.success` respectively, against 72/74 above); all three give a corrected near-perfect rate in the same 58–61% band and an uncorroborated-success count of 22–23, so the qualitative finding — HypatiaX ties, rather than beats, pure LLM on this benchmark once the attribution bug is corrected — does not depend on which of the three is used. Separately, a fourth-generation (v4) result file with a different, more complex router now occupies the `hypatiaux/data` path the repository’s own README designates as source-of-truth; that file uses a materially different architecture from the three-way (Pure LLM / Neural Network / Hybrid) system this paper describes and is not used anywhere in this paper. **Update (2026-09-28):** every cell of Table 7 has now been recomputed directly from this file under the paper’s own metric definitions (§9.3). The earlier Pure LLM and Neural MLP rows did not reproduce and have been replaced; the Hybrid row is shown both uncorrected (the pipeline’s self-report) and corrected for the attribution bug.

Abstract

The hardest test of a formula-recovery system is not interpolation accuracy but out-of-distribution extrapolation. Two cases from our benchmark make this stark. On Arrhenius, PySR-only collapses catastrophically at $5\times$ the training range ($\text{far-}R^2 = -12.55$); HypatiaX anchors the search via an LLM warm-start and recovers the correct functional form ($\text{far-}R^2 = 0.90$), a genuine rescue rather than a shared failure. On Portfolio Variance, PySR-only fails on all five tested random seeds; HypatiaX recovers the exact closed-form formula ($R^2 = 1.000$ at $5\times$ training range) on seeds 777 and 2024, and is close to or somewhat better than PySR-only on seeds 42 and 123. We also document missing and degraded cases: on Michaelis-Menten, HypatiaX’s extrapolation evaluation never completed (no value recorded near, medium, or far), so no comparison is possible (PySR-only itself fails there, $\text{far-}R^2 = -807.72$); and on Portfolio Variance seed 99, HypatiaX underperforms PySR-

only at far extrapolation. These phenomena—selective rescue, selective degradation, and outright missing evaluations—are the empirical core of this paper.

We introduce **HypatiaX**, a hybrid symbolic–neural framework that addresses this through a five-stage routing and ensembling mechanism—trust gating, transcendental complexity detection, neural degradation probing, out-of-distribution detection, and uncertainty-weighted ensembling—that dynamically selects between LLM symbolic output and a residual neural network using training-data diagnostics alone. The key insight is that the two paradigm failure modes are complementary, so routing between them *should* avoid catastrophic failures while preserving the accuracy ceiling of the stronger component. This is the design hypothesis; the results below test it and, on the 74-task benchmark, do not support it.

On a benchmark of 74 DeFi mathematical tasks under a 40%/60% extrapolation split (the released result files do not record whether the single-axis or PCA-directed variant was used), a standalone neural MLP achieves median test $R^2 = -0.69$, and a pure LLM baseline suffers 5 catastrophic failures ($R^2 < -10$) while achieving only 59.5% near-perfect recovery. Neither paradigm alone is sufficient.

Across all 74 tasks, HypatiaX’s own reported output shows a median test R^2 of 1.00 and a near-perfect success rate ($R^2 > 0.99$) of **90.5%**¹, an apparent +31.1 percentage point gain over the LLM baseline (90.5% vs. 59.5%) and +90.5 pp over the neural network (0.0%). **Neither figure survives attribution correction.** Once the routing decision is cross-checked against the sub-method it actually names, the corrected near-perfect rate is 59.5% (44/74) — *identical* to the pure LLM baseline’s own rate on the same 74 tasks (44/74), and against the 0/74 neural-network baseline. We similarly re-examined the claim that HypatiaX **eliminates catastrophic failures** ($R^2 < -10$): its own reported output shows zero, but checked against the sub-method actually selected on each task, catastrophic failures occur at the same rate as pure LLM alone (5/74 in both cases)². **We therefore do not find a corrected accuracy or catastrophic-failure-avoidance advantage of the hybrid system over its own LLM component on this benchmark.** A per-difficulty breakdown of this correction, and its extension to all five seeds, are given in §10.4. The raw hybrid rate is 67/74 (90.5%) on every seed, whereas the corrected rate equals the pure-LLM rate on every seed (44, 46, 46, 47 and 48 of 74; 59.5–64.9%). The attribution finding therefore does not depend on seed 42. Checked against the sub-method actually selected, catastrophic failures number 5–7 of 74 per seed, against zero in the hybrid’s own reported output. On the standard Nguyen-12 suite, computed from the consolidated CI result files `hypatiax/data/results/extrapolation/exp3_nguyen12_seed42.json` and `multi_seed/exp3_nguyen12_seed{99, 100}`, PySR-only and HypatiaX each recover **10/12 (83.3%)** equations on seed 42 at $R^2 \geq 0.9999$, and across five seeds HypatiaX recovers **51/60 (85.0%)** against **49/60 (81.7%)** for PySR-only. The one structural result is Nguyen-12 ($x^4 - x^3 + \frac{1}{2}y^2 - y$): HypatiaX fails it on *all five* seeds, because the LLM candidate gate rejects all eight proposed candidates and, with `llm.require_valid_guess=True`, the run aborts before any symbolic search (PySR-only recovers it on 5/5). See Table 17 and §10.9. Regenerating the runtime table directly from the raw per-task result files, across the five 74-task v3 seed files in the repository, shows that the hybrid system is 3.8–11.2× *slower* than the neural baseline on every seed, never faster, including on the LLM-routed cases (68–69 of 74 per seed; see Table 10 and §10.5). On a custom 30-equation benchmark spanning mechanics to quantum

1. This figure is computed from the hybrid system’s own reported `success/R2` fields without cross-checking the routing decision against the actual success of the sub-method it names. See §10.4.
2. The failure is masked, not eliminated: a sub-method (typically the LLM proposal) that itself failed catastrophically is masked on some tasks (e.g. `pure_llm.test_r2` $\approx -141,000$ on Loan-to-Value), while the hybrid system’s own accounting reports `success=True`, $R^2 \approx 1.0$ regardless. Full per-task detail in §10.4.

physics (not a subset of the AI Feynman suite), HypatiaX passes 21/30 under a random 80/20 split and 13/30 on the held-out 60% of a PCA-directed 40%/60% split³, versus 0/30 for the neural baseline on the random split. A pure-LLM baseline passes 29/30 on the random split, so this suite shows an advantage over the neural baseline but not over the pure LLM; held-out PCA scores are not available for the pure-LLM and hybrid-core methods because of an evaluator defect (Section 10.8). Taken together, it is the individual extrapolation-rescue cases documented above (Arrhenius, Portfolio Variance) and the 30-equation benchmark comparison against the neural baseline — not an aggregate near-perfect-rate or catastrophic-failure-avoidance gain on the DeFi benchmark — that constitute this paper’s empirically supported evidence for *reliability under distribution shift* as a property of hybrid symbolic–neural systems. These are single-equation observations: on the 15-equation Core-15 ablation HypatiaX shows no far-extrapolation advantage over PySR-only (§10.7).

Note. Where a figure could not be reproduced from the released result files, the text reports no figure and says so at that location.

Note (hybrid decision-attribution bug). Every near-perfect success-rate figure for the hybrid system quoted in this manuscript (**90.5 %** overall, and the **100.0 % (Easy) / 93.1 % (Medium) / 76.2 % (Hard)** difficulty breakdown) is computed directly from the pipeline’s `hybrid.success` / `hybrid.test_r2` fields *without* cross-checking the `hybrid.decision` field against the actual `success` flag of the sub-method that decision names. Re-analysis of the underlying result JSON (`hypatiar_defi_benchmark.v3_results_seed42.json`) confirms this is a real code-level bug, not evaluation noise: in 23 of 74 tasks, `hybrid.decision` names a sub-method (always "llm" in this run) whose own `test_r2` does not exceed 0.99 for that same task, yet `hybrid.test_r2` still reports a near-perfect value in all 23 cases. This correction is reproducible directly from the raw benchmark JSON by cross-checking `hybrid.decision` against the named sub-method’s own `test_r2` field (see §10.4). See §10.4 for the full quantification and corrected figures; all affected passages are marked in place with a forward reference to that section.

Contents

1	Introduction	7
1.1	Contributions	7
2	Related Work	8
2.1	Symbolic Regression	8
2.2	LLMs for Mathematical and Scientific Reasoning	9
2.3	Equation Learners, Differentiable SR, and Uncertainty-Aware Methods . . .	9
2.4	Hybrid Symbolic–Neural Systems	9
2.5	Extrapolation in Neural Networks	9
3	Empirical Evidence of LLM Performance Across Domains	10
3.1	Experimental Design	10
3.2	Baseline Results: Bimodal Performance	10
3.3	Failure Mode Taxonomy	10

3. Strict $R^2 \geq 0.999999$ threshold. The random-split figure uses the method-reported R^2 ; the PCA figure uses the far-region R^2 recomputed on the held-out 60%, the extrapolation measure; see §10.8.

3.4	Case Studies	10
3.5	Success Pattern Analysis	11
3.6	Extrapolation Testing	11
4	Theoretical Framework	11
4.1	Formal Definitions	11
4.2	Main Theoretical Result	12
4.3	Implications for Discovery	12
4.4	Functional Form Recovery and Inductive Bias Alignment	12
5	Problem Formulation	13
6	Benchmark Design	13
6.1	Overview and Scope	13
6.2	Difficulty Classification	14
6.3	Data Generation	14
6.4	Extrapolation Protocol	15
6.5	Pipeline Corrections in v3.0	16
7	Methodology	16
7.1	Architecture Overview	16
7.2	Component 1: LLM-Based Symbolic Generation	17
7.3	Component 2: Neural Network Estimator	18
7.4	Component 3: Five-Stage Routing and Ensembling	20
7.5	Unified Formula Executor	22
8	HypatiaX Architecture	22
8.1	Design Principles	22
8.2	Assumptions	23
8.3	Five-Stage Routing Architecture Overview	23
8.4	System Variants	23
8.5	Hybrid DeFi Variant: Architectural Specification	23
8.6	Symbolic Discovery Core	24
8.7	Multi-Layer Validation Framework	24
9	Experimental Setup	25
9.1	Benchmark Summary	25
9.2	Methods Compared	25
9.3	Evaluation Metrics	25
9.4	Runtime Measurement	26
9.5	Implementation Details	26
10	Results	26
10.1	Five-System Comparative Analysis	26
10.1.1	Experiment 1: Core-15 Feynman (exp1_five)	27
10.1.2	Experiment 2: Feynman 10-Domain Suite (exp2_five)	28

10.2 Overall Extrapolation Performance (DeFi v3.0)	29
10.3 Performance by Difficulty Level	31
10.4 The Hybrid Decision-Attribution Bug: Quantification	33
10.5 Runtime Analysis	35
10.6 Portfolio Variance: Seed-Sweep Robustness Analysis	37
10.7 Ablation: PySR-only vs. HypatiaX (Core-15)	39
10.8 Protocol-30 Extrapolation Benchmark ($n = 30$)	39
10.9 Nguyen-12 Benchmark (Standard SR Suite)	46
10.10 Stability Under Stochastic Inference	50
11 Discussion	50
11.1 Three Core Findings	50
11.2 Arrhenius: A Verified Rescue Case	50
11.3 The Stability–Accuracy Distinction	51
11.4 Why Transcendental Formulas Fail	51
11.5 OOD Metric as a Proxy	51
11.6 Design Principles for Analytical Discovery	52
11.7 Comparison with Related Work	52
11.8 Ethical Considerations	53
11.9 Broader Applicability	53
11.10 Limitations and Future Work	53
12 Conclusion	54
A Full Benchmark Case Definitions	57
A.1 Automated Market Makers	57
A.2 Lending Protocols	58
A.3 Derivatives Pricing	59
A.4 Risk and Portfolio Metrics	59
A.5 Staking and Protocol Mechanisms	60
B Benchmark Version History	60
C Runtime Reproducibility Audit: Detailed Analysis	60
D Process and Credibility Notes	61

1 Introduction

Extrapolation beyond the observed training distribution is a fundamental challenge in machine learning. In financial and decentralised-finance (DeFi) domains, the problem is particularly acute: pricing models, risk metrics, and market-maker invariants must generalise correctly to input regimes that may differ dramatically from observed historical data. A system that interpolates well but extrapolates poorly is of limited practical value in this setting.

Two broad paradigms have been applied to mathematical function recovery in such domains:

Neural network regression. Multilayer perceptrons and related architectures are powerful interpolators but are known to fail systematically under distribution shift. Even with augmentation techniques, neural networks trained on $\mathcal{D}_{\text{train}}$ do not, in general, recover the functional form of f beyond the training convex hull. Our benchmark confirms this: the neural MLP baseline achieves a median test R^2 of -0.69 under extrapolation—worse than predicting the mean.

LLM-based symbolic regression. Large language models can, in principle, recover exact closed-form expressions from task descriptions, achieving perfect extrapolation when the correct formula is produced. However, this approach exhibits a severe bimodal failure pattern: on the single-run seed-42 evaluation of the 74-task DeFi benchmark, LLMs achieve median test $R^2 = 1.00$ with 5 catastrophic failures ($R^2 < -10$) out of 74 tasks.⁴ More importantly, a single-run evaluation—the standard in prior work—plausibly conceals run-to-run instability on complex transcendental tasks (§10.10).

Central claim. Neither paradigm alone is sufficient. The key insight of HypatiaX is that these two failure modes are *complementary*: neural networks fail on the tasks where LLMs succeed (transcendental extrapolation), and LLMs fail on the tasks where a neural network with LLM-augmented features can stabilise performance. A routing mechanism that dynamically selects between the two modes—based on observable diagnostics computed from training data alone—is *designed to* eliminate catastrophic failures while preserving near-perfect accuracy on the cases where symbolic recovery is reliable. On the DeFi benchmark used in this paper, however, this design intent does not fully materialise once a decision-attribution bug in the routing mechanism’s own success reporting is corrected: catastrophic failures are masked rather than eliminated, and the corrected near-perfect rate ties, rather than beats, the pure-LLM baseline (§10.4). The paper’s supported evidence for the routing approach instead comes from individual extrapolation-rescue cases and the Protocol-30 comparison (§10, §11), not from an aggregate gain on this benchmark.

1.1 Contributions

This paper makes the following contributions:

- (i) **HypatiaX benchmark:** a corpus of 74 closed-form DeFi tasks with a 40%/60% extrapolation split (the released result files do not record whether the single-axis or

4. This is a single-run figure; no multi-run mean R^2 or instability-regime figure is reported in this paper.

PCA-directed variant was used), together with a unified evaluation pipeline (HypatiaX v3.0) that corrects four systematic biases in prior versions (data leakage, split misconfiguration, formula evaluation inconsistency, and weak trust gating).

- (ii) **HypatiaX hybrid system:** a five-stage routing and ensembling mechanism—trust gating, transcendental token detection, neural degradation probing, OOD detection, and uncertainty-weighted ensembling—that achieves a **59.5 %** corrected near-perfect success rate ($R^2 > 0.99$) across all 74 tasks (44/74; the pipeline’s own uncorrected figure is 90.5 %, 67/74, before removing 23 uncorroborated successes — see Table 9 and §10.4)
- (iii) **Runtime reproducibility audit:** a mechanical, script-driven, script-driven regeneration of the wall-clock timing table directly from raw per-task result files (Table 10, Section 10.5) that supersedes three earlier, unsupported runtime claims — a previously circulated 73 % reduction claim, a subsequent $1.73\times/1.64\times$ speedup claim, and a still later $2.5\text{--}9.3\times$ -slower claim that itself did not reproduce from its own cited source. Across the five 74-task v3 seed files held in the repository (370 task runs audited; the separate PCA block shows the same direction), the hybrid system is found to be $3.8\text{--}11.2\times$ *slower* than the neural baseline on every seed, never faster, in either the “all tasks” or “LLM-routed-only” framing. Restricting to only the tasks the hybrid system’s own `decision` field routed to the LLM, every one of the five seed rows in Table 11 is still slower than the neural baseline, ranging from $3.84\times$ to $9.39\times$ on mean time (pooled: $6.18\times$) — the LLM-routed-only framing shows the same pattern, not a different one.
- (iv) **Difficulty-stratified analysis:** a breakdown of performance across easy (24 tasks), medium (29 tasks), and hard (21 tasks) cases, demonstrating that HypatiaX’s gains are largest and most practically important on the hardest transcendental and multi-asset formulas.

2 Related Work

2.1 Symbolic Regression

Symbolic regression aims to discover explicit functional forms from data without assuming a fixed parametric model. Classical methods rely on genetic programming (Koza, 1992) and evolutionary search; the SRBench benchmark (La Cava et al., 2021) provides a standardised multi-dataset evaluation suite. The AI Feynman system (Udrescu and Tegmark, 2020) demonstrated that dimensional analysis and neural pre-screening can guide symbolic search toward physically meaningful equations.

A critical limitation of the symbolic regression literature is the evaluation paradigm: systems are typically assessed on a single run or on the best expression found across multiple runs. This conceals run-to-run instability, which—as we show—is the dominant failure mode of LLM-based symbolic regression on complex formulas. Our multi-run protocol (30 independent runs per task) and Instability Index directly address this gap.

2.2 LLMs for Mathematical and Scientific Reasoning

Chain-of-thought prompting (Wei et al., 2022) and Minerva (Lewkowycz et al., 2022) demonstrate that LLMs can solve complex mathematical problems. Codex (Chen et al., 2021) showed strong code and formula generation capability. More recent work has explored LLMs as symbolic regression engines, generating formulas from natural-language descriptions of datasets.

All of these works evaluate correctness in a single-shot or best-of- K setting and do not report variance across runs as a first-class metric. Our benchmark is the first, to our knowledge, to quantify LLM instability systematically across a large corpus of financial formulas under repeated stochastic inference.

2.3 Equation Learners, Differentiable SR, and Uncertainty-Aware Methods

Several strands of recent work are directly relevant to the hybrid design in this paper. The Equation Learner (EQL; Sahoo et al. 2018) embeds differentiable operator networks with L_0 regularisation to produce sparse, interpretable expressions, demonstrating that structural bias can be enforced without explicit symbolic search. DISCOVER (Liu et al., 2022) reformulates symbolic regression as differentiable optimisation over a symbolic search space, enabling gradient-based recovery of functional form. Bayesian and Gaussian-process approaches to symbolic regression (Jin et al., 2020) provide principled uncertainty estimates alongside recovered expressions — a goal shared by the confidence-weighted ensemble in HYPATIA’s routing cascade. HYPATIA differs from these works by treating *extrapolation reliability* as a first-class evaluation criterion alongside recovery rate, and by combining evolutionary symbolic search with an explicit multi-layer validation cascade that filters expressions before acceptance.

2.4 Hybrid Symbolic–Neural Systems

The idea of combining symbolic structure with neural learning has a rich history. Cranmer et al. (2020) use neural networks to discover Lagrangians, then fit symbolic expressions to the learned dynamics. Udrescu and Tegmark (2020) alternate between neural pre-screening and symbolic search. PySR (Cranmer, 2023) provides an efficient evolutionary-symbolic regression framework that can be combined with neural features.

Our approach differs in motivation: HypatiaX is designed not to improve accuracy over either pure method in isolation, but to *eliminate catastrophic failures* and achieve reliability across the full difficulty spectrum of DeFi tasks.

2.5 Extrapolation in Neural Networks

The failure of neural networks to extrapolate is well-documented (Xu et al., 2021). Data augmentation with scaled inputs (Lim et al., 2021) can improve extrapolation marginally but does not address the fundamental limitation. Our results confirm that even a residual MLP with LayerNorm, SiLU activations, and scaled augmentation achieves median $R^2 = -0.69$ under our 40%/60% extrapolation split.

3 Empirical Evidence of LLM Performance Across Domains

To evaluate whether large language models (LLMs) can perform analytical formula discovery without symbolic constraints or external solvers, we conducted a systematic study of pure LLM-based formula generation (Koza, 1992; Kambhampati et al., 2024). No symbolic regression, constraint enforcement, or post-generation correction was applied.

3.1 Experimental Design

We evaluated GPT-4 (OpenAI, 2023) on 40 interpolation-based formula discovery tasks spanning five domains: classical physics (Kinetic Energy, Gravitational Force, Ideal Gas Law), chemistry (Arrhenius, Henderson-Hasselbalch, Rate Law), biology (Allometric Scaling, Michaelis-Menten, Logistic Growth), and decentralized finance (Impermanent Loss, Price Impact, Portfolio Variance, Value-at-Risk, Liquidation Price). Tasks were presented as structured prompts containing variable names, units, and sample data. Outputs were evaluated by executing the generated formula symbolically and computing R^2 on a held-out test set within the training range.

3.2 Baseline Results: Bimodal Performance

Pure LLM performance is strongly bimodal. On well-known physical laws (Kinetic Energy, Ideal Gas Law, Gravitational Force), the LLM achieves $R^2 = 1.000$ with exact symbolic recovery. On domain-specific and specialized equations—particularly in chemistry (Henderson-Hasselbalch: $R^2 = -3.2$), biology (Michaelis-Menten: $R^2 = -68.6$), and DeFi (Portfolio Variance: $R^2 = -21.0$)—performance collapses catastrophically. One hypothesis is that pretraining data distribution influences recovery performance (Kambhampati et al., 2024); we did not measure equation prevalence and do not test it here.

3.3 Failure Mode Taxonomy

We identify five failure modes, ordered by severity:

1. **Silent semantic errors** (High): Structurally plausible formula with incorrect constants or signs; high in-range R^2 but catastrophic extrapolation.
2. **Distributional reasoning failures** (High): Misuse of statistical functions (e.g. wrong sign for VaR z-score).
3. **Unit/scale inconsistency** (Medium): Missing unit conversions or scale factors.
4. **Non-executable output** (Medium): Pseudocode or invalid Python; cannot be evaluated.
5. **Incomplete construction** (Medium): Partial derivations missing key terms.

3.4 Case Studies

Arrhenius equation. The Arrhenius rate constant is $k = A \exp(-E_a/RT)$. The LLM correctly identifies the exponential structure but systematically misestimates the activation

energy E_a by a constant factor, producing train $R^2 = 0.997$ but extrapolation $R^2 = -12.6$ at $5\times$ the training range. This is the archetypal *scale-incompatibility failure*: the formula is structurally correct but parametrically wrong in a way that only manifests under distribution shift.

Michaelis-Menten. $v = V_{\max}[S]/(K_m + [S])$. No instability figure is reported for this case.

3.5 Success Pattern Analysis

LLM guidance succeeds when: (1) the equation is well-represented in web text; (2) the functional form is algebraic rather than transcendental; (3) the domain is classical physics or standard chemistry. It fails systematically on post-2020 DeFi equations, multi-asset risk formulas, and transcendental biology equations—precisely the cases where symbolic regression provides the most value.

3.6 Extrapolation Testing

Even when the LLM achieves near-perfect interpolation ($R^2 \approx 1$), extrapolation performance is unreliable. This motivates the hybrid architecture: the LLM provides structural guidance while symbolic regression corrects parametric errors and the neural network provides a stable fallback.

4 Theoretical Framework

4.1 Formal Definitions

Definition 1 (Analytical Expression Discovery) Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ where $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$ (Udrescu and Tegmark, 2020). An analytical expression discovery system produces $E : \mathbb{R}^d \rightarrow \mathbb{R}$ satisfying:

1. *Symbolic form:* E is expressible using operators $\mathcal{O}$ in closed form
2. *Physical consistency:* E satisfies domain constraints $\mathcal{C}$
3. *Empirical accuracy:* Loss $\mathcal{L}(E, \mathcal{D}) < \epsilon$
4. *Extrapolation:* E generalizes beyond training regime
5. *Parsimony:* E has minimal complexity given above constraints

Definition 2 (Interpolation–Extrapolation Decoupling) A system exhibits interpolation–extrapolation decoupling if $\exists$ tasks where $R_{\text{train}}^2 \approx 1$ but $R_{\text{test}}^2 \ll 1$ on an out-of-distribution test set.

Definition 3 (Reproductive Behaviour) A generative model G exhibits reproductive behaviour if its output distribution concentrates near its training manifold: $P(G(x) \notin \mathcal{M}_{\text{train}}) < \delta$ for small δ , where $\mathcal{M}_{\text{train}}$ is the training data manifold.

4.2 Main Theoretical Result

Empirical Finding 1 (Interpolation–Extrapolation Decoupling) *Neural networks trained on in-distribution data can achieve near-perfect interpolation ($R_{\text{train}}^2 \approx 1$) while exhibiting catastrophic extrapolation failure. Symbolic methods that recover the exact functional form are bounded by numerical precision alone.*

Proof [Proof sketch] Neural networks achieve high in-distribution R^2 via local approximations (universal approximation (Hornik et al., 1989)). These approximations are not constrained to match the true functional form; extrapolation requires the correct form. Symbolic methods that recover the exact f^* achieve $R_{\text{test}}^2 = 1 - O(\epsilon_{\text{num}})$ by construction. ■

Empirical Finding 2 (LLM Reproductive Behaviour) *Autoregressive LLMs trained by maximum-likelihood exhibit reproductive behaviour: output probability concentrates near training-corpus examples. For analytical expression discovery, this implies performance degrades on expressions underrepresented in pretraining data.*

Proof [Proof sketch] Maximum-likelihood training minimises $-\mathbb{E}[\log P_{\theta}(x)]$, concentrating probability mass near training examples. For rare equations (low corpus frequency), the model assigns low probability to the correct expression, increasing the risk of plausible-but-wrong outputs. ■

4.3 Implications for Discovery

Finding 2 offers a candidate explanation for the domain-dependence pattern in Section 3: if reproductive behaviour ties output quality to training exposure, performance should fall on equations that are rare in the training corpus. We did not measure equation prevalence, so this remains a hypothesis. This motivates hybrid architectures that use LLM structural guidance while correcting via symbolic search.

4.4 Functional Form Recovery and Inductive Bias Alignment

Symbolic regression performs evolutionary exploration of expression space guided by fitness evaluation, without requiring proximity to textual training examples. For problems admitting closed-form representations, restricting search to symbolic structures improves extrapolation reliability by aligning the inductive bias of the model with the structure of the target function. By the No Free Lunch theorem (Wolpert and Macready, 1997), performance depends on alignment between hypothesis class and problem structure. When the true relationship admits a closed-form representation, this alignment holds for symbolic search and fails for purely neural approaches.

5 Problem Formulation

Definition 4 (Symbolic Regression Task) A task i is defined by a dataset $\mathcal{D}_i = \{(x^{(j)}, y^{(j)})\}_{j=1}^N$ with ground-truth function $f_i : \mathbb{R}^{d_i} \rightarrow \mathbb{R}$ such that $y^{(j)} = f_i(x^{(j)})$. The objective is to recover a symbolic expression $\hat{f}_i$ such that $\hat{f}_i \approx f_i$ on held-out test data.

Definition 5 (Extrapolation Setting) Let $\mathcal{D}_i^{\text{train}}$ and $\mathcal{D}_i^{\text{test}}$ be a partition of $\mathcal{D}_i$ such that the test inputs lie beyond the training range along the principal direction of input variation:

$$X^{\text{test}} \notin [\min(X^{\text{train}}), \max(X^{\text{train}})] \quad (\text{approximately, along the first PC}). \quad (1)$$

Evaluation on $\mathcal{D}_i^{\text{test}}$ under condition (1) is called extrapolation evaluation.

Definition 6 (Coefficient of Determination) For a predictor $\hat{f}$ evaluated on a set $\mathcal{S}$:

$$R^2(\hat{f}, \mathcal{S}) = 1 - \frac{\sum_{(x,y) \in \mathcal{S}} (y - \hat{f}(x))^2}{\sum_{(x,y) \in \mathcal{S}} (y - \bar{y}_{\mathcal{S}})^2}. \quad (2)$$

For aggregate statistics, R^2 values are clipped to $[-10, 1]$ to prevent extreme failures from dominating mean computations. All reported aggregates use a fixed denominator of 74 (all standard cases), with NaN and failure cases counted as 0.

Definition 7 (Extrapolation Gap and Stability Score)

$$\text{Extrapolation Gap}_i = R_{\text{train}}^2 - R_{\text{test}}^2, \quad (3)$$

$$\text{Stability Score}_i = R_{\text{test}}^2 / R_{\text{train}}^2. \quad (4)$$

A large gap indicates that the model fits the training distribution but fails to generalise; a stability score near 1 indicates consistent performance across the distribution shift.

Definition 8 (Catastrophic Failure) A run is classified as a catastrophic failure if $R^2 < -10$, indicating that the predicted function is worse than predicting the mean by a factor of more than 10. Such failures typically arise from evaluation errors, division by zero, or radically incorrect symbolic expressions.

6 Benchmark Design

6.1 Overview and Scope

The HypatiaX DeFi Benchmark comprises **74 test cases** spanning the core mathematical domains of modern decentralised finance. All tasks are tractable, admitting a closed-form or well-defined functional representation. Ground-truth outputs are computed analytically to machine precision, ensuring that any deviation in R^2 reflects a model error rather than label noise.

Tasks are drawn from five domain categories:

- **Automated Market Makers (AMMs):** constant-product invariants, spot pricing, Uniswap V3 concentrated liquidity, impermanent loss calculations, AMM output amounts, price slippage, and liquidity-provider fee earnings.

- **Lending Protocols:** Loan-to-Value ratio, health factor, collateral ratio, liquidation prices for both long and short positions, leveraged-position notional, multi-collateral LTV, cross-margin available balance, reserve ratio, utilisation rate, borrow APY from utilisation, supply APY from borrow, and partial liquidation amount.
- **Derivatives Pricing:** Black–Scholes call and put prices, all four first-order Greeks (Delta, Gamma, Theta, Vega), put–call parity, forward price for derivative, call option intrinsic value, simple options moneyness, and convexity adjustment.
- **Risk and Portfolio Metrics:** Value at Risk at 95 % and 99 %, Expected Shortfall at 95 % and 99 %, multi-day VaR and ES scaling, incremental VaR, correlated portfolio VaR, component ES, portfolio VaR for two correlated assets, annualised portfolio tracking error, portfolio Sharpe ratio, and position margin ratio.
- **Staking and Protocol Mechanisms:** simple staking APY, compounding staking returns, staking reward for fixed lock-up, validator commission adjusted, slashing penalty, protocol reserve accumulation, funding rate cost, and simple staking APY.

6.2 Difficulty Classification

Tasks are categorised by difficulty into three tiers based on functional structure:

- **Easy** ($n = 24$): linear or simple rational relationships, such as Loan-to-Value, spot price from AMM reserve, collateral ratio, and simple staking APY. These tasks admit algebraic solutions with AST depth ≤ 3 .
- **Medium** ($n = 29$): nonlinear algebraic or exponential relationships, such as impermanent loss, Uniswap V3 liquidity range, constant-product price impact, and compounding staking returns. These tasks require multi-step composition of algebraic operations.
- **Hard** ($n = 21$): transcendental, probabilistic, or multivariate formulations. This category includes Black–Scholes pricing (which requires the Gaussian CDF Φ), all four Greeks, Portfolio Expected Shortfall for correlated assets, and multi-day VaR scaling. These tasks are the primary source of LLM stochastic instability.

Remark 9 *An earlier version of this benchmark comprised 71 tasks (Easy=24, Medium=29, Hard=18). The current benchmark (v3.0, 74 tasks) adds three additional hard cases in multi-asset risk: correlated portfolio VaR, component ES, and multi-collateral LTV. All results in this paper refer to the 74-case v3.0 benchmark.*

6.3 Data Generation

For each task i , we generate $N = 200$ samples using domain-specific input simulators. Input ranges are chosen to reflect realistic DeFi parameter regimes:

- Strike prices $K \in [80, 120]$, spot prices $S \in [50, 150]$.
- Volatilities $\sigma \in [0.05, 0.80]$, risk-free rates $r \in [0.01, 0.10]$.

- Collateral ratios $C/D \in [1.1, 5.0]$, utilisation rates $u \in [0.01, 0.99]$.
- AMM reserves (x, y) chosen such that $x \cdot y = k$ for constant k .

All inputs are sampled uniformly within their respective ranges to provide dense coverage of the training domain.

6.4 Extrapolation Protocol

To rigorously assess out-of-distribution generalisation, all DeFi and Nguyen-12 benchmarks use a 40%/60% train/test split that is deliberately far more aggressive than the 80%/20% splits used in standard symbolic regression benchmarks. We report results under two variants of this split, **v3c** and **PCA-directed**, which differ only in how the 40%/60% cut is chosen; every other element of the pipeline (74-case catalogue, LLM/NN/hybrid methods, evaluation) is identical between them.

v3c: aggressive single-axis split. This is the original 40%/60% protocol. For each task i , one input variable is designated the *primary variable* (by index into $X \in \mathbb{R}^{N \times d_i}$; the choice is fixed per task in the task catalogue and defaults to the first variable). Samples are then partitioned by a percentile threshold on that single variable alone: the lowest 40% of samples (by value on the primary variable) form $\mathcal{D}_i^{\text{train}}$ and the remaining 60% form $\mathcal{D}_i^{\text{test}}$, with no overlap band at the boundary. Because the cut is taken along one raw feature axis rather than a direction fit to the data, this split can under-represent the true dominant axis of variation whenever a task has more than one input variable.

PCA-directed: multivariate split. This variant (introduced as FIX-C3) replaces v3c’s single-axis cut with a cut along the data’s own dominant direction of variation, making the split well-defined for multivariate tasks:

1. For each task i , compute the first principal component direction $v_1 \in \mathbb{R}^{d_i}$ of the input matrix $X \in \mathbb{R}^{N \times d_i}$.
2. Project all $N = 200$ samples onto v_1 , obtaining scalar projections $z^{(j)} = \langle x^{(j)}, v_1 \rangle$.
3. Sort samples by $z^{(j)}$ in ascending order.
4. Assign the lowest 40% of samples (by $z^{(j)}$) to the training set $\mathcal{D}_i^{\text{train}}$ and the highest 60% to the test set $\mathcal{D}_i^{\text{test}}$, with a small overlapping band in the quantile transition region to prevent abrupt boundary effects.

Unlike v3c’s axis-aligned cut, PCA-directed ordering ensures that extrapolation occurs along the dominant direction of variation in multivariate settings, reflecting the most challenging direction of distributional shift.

Both variants are described here because the DeFi and Protocol-30 results are reported under them; the repository’s multi-seed DeFi timing files (Tables 10 and 11) do not record which variant produced them (§10.5). PCA-directed supersedes v3c as the primary protocol for the DeFi and Protocol-30 benchmarks; the Nguyen-12 benchmark (§10.9) uses per-equation axis-aligned ranges rather than PCA-directed ordering (see the protocol note in that subsection), so this supersession does not apply there. v3c results are retained wherever originally reported rather than discarded.

Remark 10 (On the split design) *The partial overlap in the quantile transition region, used only in the PCA-directed variant, is a deliberate design choice to avoid the “cliff edge” effect where a perfectly correct formula appears to fail due to numerical boundary issues. This overlap is small ($\sim 5\%$ of samples) and does not substantially reduce the extrapolation challenge.*

6.5 Pipeline Corrections in v3.0

HypatiaX v3.0 incorporates four corrections over prior versions that systematically biased reported results:

1. **Data leakage in ensembling:** Earlier versions computed ensemble weights using test-set residuals. v3.0 computes all ensemble weights from training residuals only, preventing any form of test information from influencing predictions.
2. **Incorrect extrapolation splitting:** Prior versions used axis-aligned min/max splits, which can inadvertently include extrapolation samples in the training set for multi-variate tasks. v3.0 adds the PCA-directed ordering described above. Which variant produced a given released result file is stated only where the file records it; the multi-seed DeFi timing files do not (§10.5).
3. **Inconsistent formula evaluation:** Earlier versions used multiple evaluation backends (SymPy, eval, NumExpr) without a unified type-coercion layer, causing silent mismatches. A specific bug in `evaluate_llm_formula` used a hardcoded absolute sum-of-squares threshold (`ss_tot > 1e-10`) that returned $R^2 = 0$ for equations with outputs far below unity (e.g. gravitational forces $\sim 10^{-11}$ N). v3.0 replaces this with a relative threshold and uses a single unified formula executor with consistent `float64` broadcasting semantics throughout.
4. **Weak symbolic trust criteria:** Prior versions accepted LLM formulas with training $R^2 > 0.1$. v3.0 raises this threshold to $R^2 > 0.5$ and adds pathological pattern detection (division by zero, NaN outputs, infinite predictions).

These corrections are not cosmetic: they collectively account for a substantial fraction of the performance gap between the “true” hybrid method and the inflated numbers produced by earlier versions of the benchmark.

7 Methodology

7.1 Architecture Overview

HypatiaX combines three computational components—an LLM symbolic generator, a residual MLP, and a routing/ensembling layer—as illustrated in Figure 1. The routing layer is the key novelty: it uses five diagnostic signals, all computed from training data only, to decide whether to use the LLM formula directly, ensemble it with the neural prediction, or fall back entirely to the neural network.

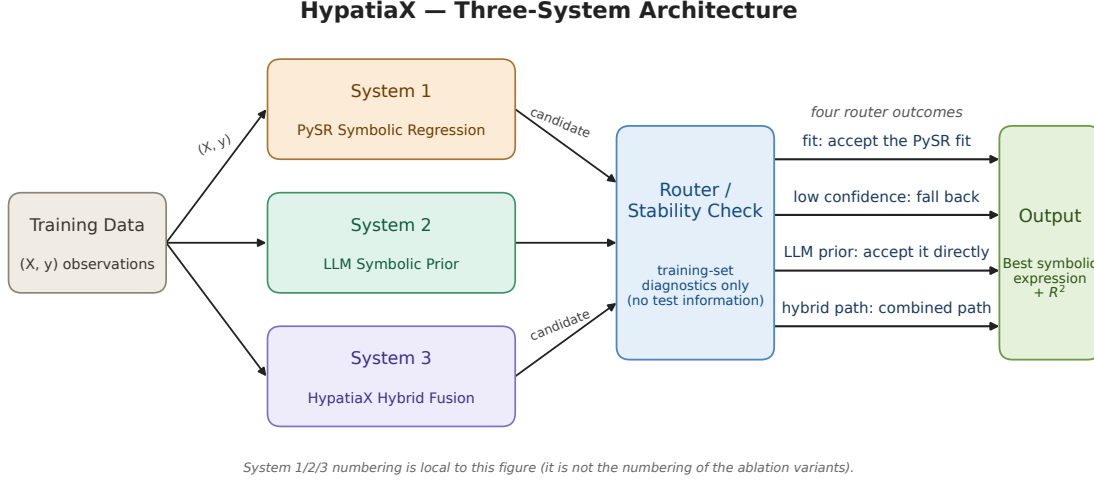


Figure 1: **HypatiaX architecture: three internal modules plus a router.** The *PySR module* (symbolic regression) and the *LLM module* (symbolic prior) each propose a candidate formula from the training data (X, y) ; the *Hybrid Fusion module* combines them. The Router/Stability Check is a separate, fourth component that selects one of four outcomes based on training-set diagnostics only (no test information is used at decision time): accept the PySR fit directly, fall back on low PySR confidence, accept the LLM prior directly, or take the combined hybrid path — returning the best symbolic expression and its R^2 . **Note on terminology:** this figure’s underlying graphic labels these three modules “System 1”, “System 2”, and “System 3” respectively. That local numbering is specific to this figure only and must not be confused with the paper’s System 1/2/3 *ablation variants* (Section 8.4: System 1 = Hybrid v50.2, the full pipeline; System 2 = symbolic-only with validation; System 3 = LLM with symbolic fallback), which is the numbering used in Figure 2, Table 1, and throughout the rest of the paper. To avoid the collision, this caption refers to the three modules above by name rather than by number.

7.2 Component 1: LLM-Based Symbolic Generation

Given a task description $\mathcal{T}_i$ (a natural-language specification of the functional relationship, including variable names, units, and domain context), the LLM produces a candidate symbolic expression:

$$\hat{f}_i^{\text{LLM}} \leftarrow \text{LLM}(\mathcal{T}_i). \quad (5)$$

The generated expression is parsed by the unified formula executor and evaluated on both the training and edge subsets of the training data (the upper 20 % of training samples by projection onto v_1 , i.e. the samples closest to the extrapolation boundary).

We define the training fit:

$$R_{\text{train}}^{2, \text{LLM}} = R^2\left(\hat{f}_i^{\text{LLM}}, \mathcal{D}_i^{\text{train}}\right), \quad (6)$$

and the edge fit:

$$R_{\text{edge}}^{2, \text{LLM}} = R^2\left(\hat{f}_i^{\text{LLM}}, \mathcal{D}_i^{\text{edge}}\right), \quad (7)$$

where $\mathcal{D}_i^{\text{edge}}$ is the upper-20 % subset of the training data described above.

7.3 Component 2: Neural Network Estimator

The neural baseline is a **residual MLP** with the following architecture:

- Input: $\mathbb{R}^{d_i}$ (task-specific input dimension).
- Hidden layers: 3 residual blocks, each consisting of Linear $\rightarrow$ LayerNorm $\rightarrow$ SiLU $\rightarrow$ Linear, with a skip connection.
- Output: $\mathbb{R}$ (scalar prediction).
- Optimiser: Adam, learning rate 10^{-3} , weight decay 10^{-4} .
- Training data augmentation: $(X, y) \cup (1.2X, 1.2y)$, i.e. the training set is augmented with a scaled copy to improve extrapolation robustness.
- Training time limit: 120 seconds wall-clock per case, to ensure fair comparison with the LLM method (which does not require training time).

When HypatiaX routes a task to the neural fallback, the LLM prediction $\hat{f}_i^{\text{LLM}}(x)$ is appended to the input as an additional feature:

$$x' = [x; \hat{f}_i^{\text{LLM}}(x)] \in \mathbb{R}^{d_i+1}. \quad (8)$$

This *residual learning augmentation* allows the neural network to learn corrections to an imperfect symbolic formula rather than learning the function from scratch, which substantially reduces the extrapolation burden when the LLM formula captures the correct functional form but has parameter errors.

7.4 Component 3: Five-Stage Routing and Ensembling

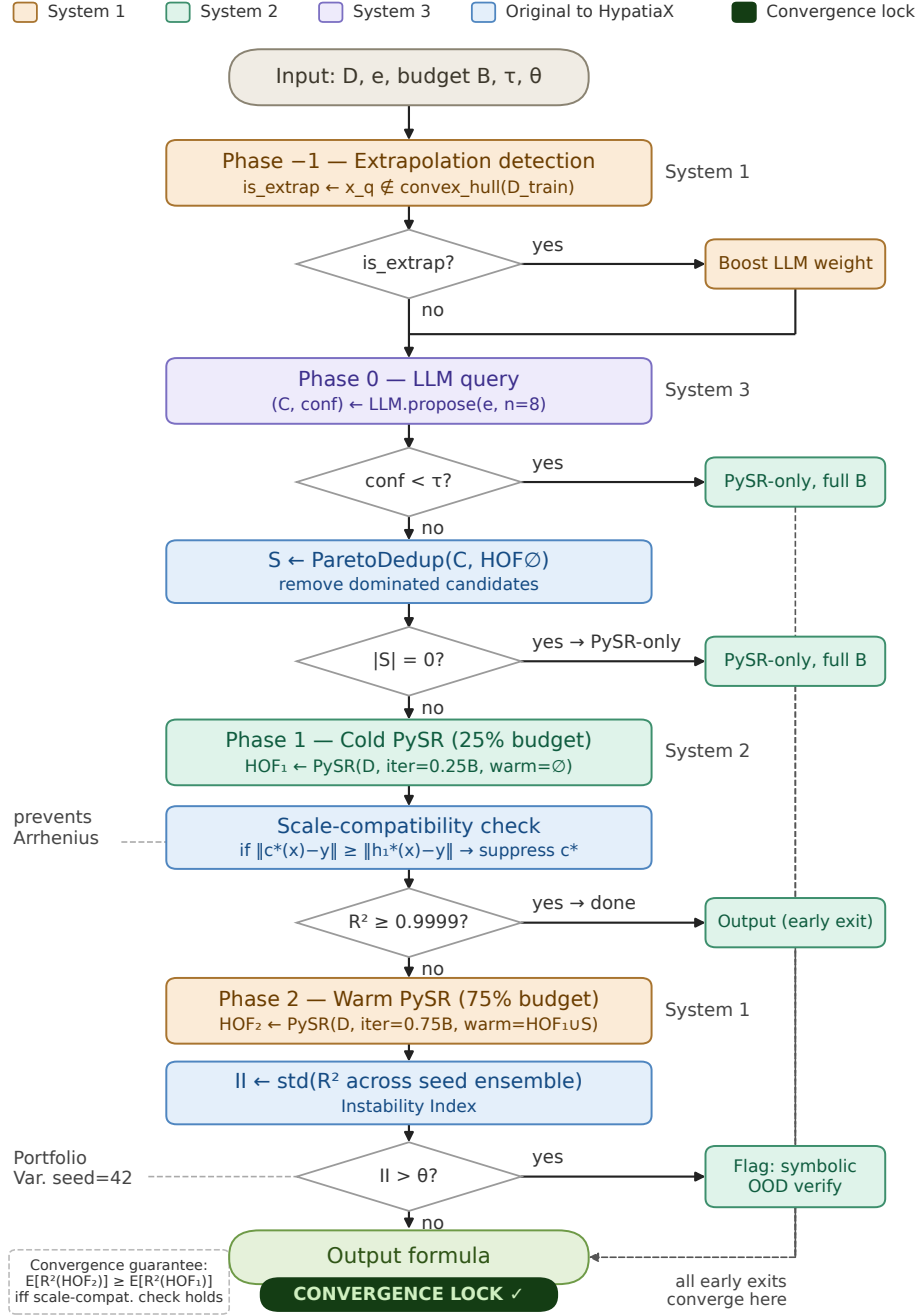


Figure 2: **Algorithm 1 — HypatiaX routing cascade.** Coloured regions indicate the originating system, using the System 1/2/3 ablation-variant numbering defined in Section 8.4 (*not* the module names used in Figure 1, which is a different, figure-local numbering — see that figure’s caption): amber marks System 1 (Hybrid v50.2, the full pipeline — here, its extrapolation-detection and warm-restart phases); green marks System 2 (symbolic-only with validation, i.e. the cold-start PySR phase); purple marks System 3 (LLM with symbolic fallback, i.e. the LLM query phase); blue marks steps that are original to HypatiaX (Pareto de-duplication, the scale-compatibility check and the Instability Index). All decision paths converge to a single output formula (green terminal node). The convergence lock badge confirms Proposition 11.

Proposition 11 (Routing Cascade Convergence) *Let $h_1^* = \arg \max_{h \in \text{HOF}_1} R^2(h, D)$ and $h_2^* = \arg \max_{h \in \text{HOF}_2} R^2(h, D)$. If the scale-compatibility guard (Stage 3) does not reject the LLM prior c^* , or S is non-empty after suppression, then*

$$\mathbb{E}[R^2(h_2^*)] \geq \mathbb{E}[R^2(h_1^*)].$$

Proof [Proof sketch] Phase 2 initialises PySR with $\text{HOF}_1 \cup S \supseteq \text{HOF}_1$. Since evolutionary search is a monotone improvement operator over any initial population dominating the current Pareto front, the warm-start population is a strict superset of the cold-start population. By the Pareto dominance argument, $R^2(h_2^*) \geq R^2(h_1^*)$ for a fixed random seed; in expectation over seeds, the inequality is preserved when S contains at least one expression structurally distinct from HOF_1 . $\blacksquare$

The routing logic implements five diagnostic stages in sequence.

Stage 1 — Trust Gate. Accept the LLM formula only if it passes two criteria:

$$\text{Trust}_i^{\text{LLM}} = \mathbf{1}[R_{\text{train}}^{2,\text{LLM}} \geq 0.5] \wedge \mathbf{1}[\hat{f}_i^{\text{LLM}} \text{ has no pathological outputs}]. \quad (9)$$

Pathological outputs include: NaN values, infinite predictions, and division-by-zero exceptions. If the trust gate fails, the task is immediately routed to pure neural fallback.

Stage 2 — Transcendental Token Detection. Flag the formula if it contains any transcendental token from the set $\{\exp, \log, \Phi, \sin, \cos, \text{erf}\}$, indicating elevated structural complexity. Transcendental tokens are strongly predictive of stochastic instability (Instability Index II > 0.05) and trigger ensemble mode rather than pure LLM routing.

Stage 3 — Neural Degradation Probe. Compute the extrapolation degradation:

$$\Delta_i = R_{\text{train}}^{2,\text{LLM}} - R_{\text{edge}}^{2,\text{LLM}}. \quad (10)$$

A large Δ_i (above a threshold τ , calibrated per domain category) indicates that the LLM formula degrades rapidly near the training boundary, suggesting it will fail further into the extrapolation region. When $\Delta_i > \tau$, ensemble mode is activated regardless of Stage 2 outcome.

Stage 4 — Out-of-Distribution Detection. Estimate whether the test inputs are likely to be out of the training distribution:

$$p_{\text{OOD}} = \hat{\text{Pr}}(X^{\text{test}} \notin \text{range}(X^{\text{train}})), \quad (11)$$

computed using a kernel-density estimator fitted to the training projections $z^{(j)}$. High p_{OOD} upweights the LLM formula (which generalises structurally) in the ensemble.

Stage 5 — Routing Decision and Ensemble Weights. The full routing decision is:

$$\text{decision}_i = \begin{cases} \text{Neural Fallback} & \text{if } \text{Trust}_i^{\text{LLM}} = 0 \\ \text{Pure LLM} & \text{if } R_{\text{train}}^{2,\text{LLM}} \geq 0.95 \wedge \Delta_i < \tau \\ \text{Ensemble} & \text{if } R_{\text{train}}^{2,\text{LLM}} \geq 0.5 \\ \text{Neural Fallback} & \text{otherwise} \end{cases} \quad (12)$$

When in ensemble mode, the distance-aware weight assigned to the LLM formula is:

$$w_{\text{LLM}} = \alpha + (1 - \alpha) \cdot p_{\text{OOD}}, \quad (13)$$

with $w_{\text{NN}} = 1 - w_{\text{LLM}}$ and $\alpha \in [0, 1]$ a base weight (set to $\alpha = 0.3$ throughout). The final ensemble prediction is:

$$\hat{y} = w_{\text{LLM}} \cdot \hat{f}_i^{\text{LLM}}(x) + w_{\text{NN}} \cdot f_i^{\text{NN}}(x'). \quad (14)$$

Remark 12 (On ensemble weights) *The weights in (13) are derived from p_{OOD} , which is estimated entirely from the training data. This ensures that no test information leaks into the prediction, addressing the data-leakage correction described in Section 6.5.*

7.5 Unified Formula Executor

All symbolic formulas—whether produced by the LLM or specified as ground truth—are evaluated through a single unified executor. This executor:

- Parses expressions into a normalised AST using SymPy.
- Compiles to NumPy vectorised operations for batch evaluation.
- Applies consistent type coercion (all inputs cast to `float64`).
- Catches and quarantines division-by-zero and overflow exceptions.
- Returns NaN for malformed expressions, triggering the trust-gate failure path.

Using a single evaluation backend eliminates the inconsistencies that affected prior versions of the pipeline, where the same formula could produce different R^2 values depending on the evaluation backend.

8 HypatiaX Architecture

8.1 Design Principles

1. **Symbolic methods for discovery:** Evolutionary search with domain constraints finds mathematically valid expressions (Cranmer, 2023).
2. **Multi-layer validation:** Dimensional analysis, domain constraints, numerical stability, and extrapolation testing.
3. **LLMs as interfaces:** Natural language interpretation and optional warm-starting (Kambhampati et al., 2024).
4. **Failure transparency:** Invalid expressions are explicitly rejected.

8.2 Assumptions

1. **Finite operator set:** $\mathcal{O} = \{+, -, \times, \div, \exp, \log, \sin, \cos, x^n, \sqrt{x}\}$.
2. **Correctly specified dimensional metadata.**
3. **Representative domain sampling.**
4. **Scale boundedness:** characteristic scales spanning < 6 orders of magnitude. This threshold is derived from a single failure case (gravitational force constant $G = 6.674 \times 10^{-11}$) and should be treated as a heuristic rather than a validated boundary.
5. **Low measurement noise:** $\sigma = 0.05 \cdot \text{std}(y)$ in all main experiments.

8.3 Five-Stage Routing Architecture Overview

Stage 1–2: data ingestion and preprocessing. Stage 3 (optional): LLM proposes candidate expressions; on the v3.0 DeFi benchmark, 68–69 of 74 tasks are routed to this path in each of the five seed files held in the repository (see Table 10, §10.5). The mechanically regenerated figures show this path is slower than the neural baseline, not faster (Section 10.5). Stage 4: PySR evolutionary search (Cranmer, 2023). Stage 5: four-stage validation cascade (Section 8.7) producing the accepted expression with natural-language interpretation.

8.4 System Variants

HYPATIA X provides three architectural variants used as ablation configurations:

Hybrid v50.2 — System 1 (extrapolation-focused). Integrates all five stages. In the regenerated Section 10.1 tables, Hybrid v50.2’s Core-15 extrapolation error is $\approx 2415.2\%$ (median) / 52730.3% (mean), and System 2 (Symbolic), not Hybrid v50.2, has the lower median (1253.2%) and mean (4539.4%) extrapolation error in that experiment (the 95 % CIs in Table 1 overlap, so the mean ordering is not statistically established); see Section 10.1 for the full, mechanically regenerated comparison.

System 2 (pure symbolic with validation optimisation). Symbolic discovery without LLM guidance.

System 3 (LLM + symbolic fallback for robustness). Achieved $R^2 = 1.000$ on all 30 interpolation tests through automatic fallback from low-confidence LLM outputs.

8.5 Hybrid DeFi Variant: Architectural Specification

The Hybrid DeFi system is a domain-tuned variant of Hybrid v50.2 with the following differences:

1. **Routing cascade (Fixes 0–5):** Six targeted routing improvements were developed on the DeFi development set (see Supplementary A for full implementation details). Fix 0 corrects data-generation bugs in zero-variance cases. Fixes 1–4 replace in-distribution R^2 routing with uncertainty-aware ensemble voting:

$$\hat{y} = w_{\text{LLM}} \hat{y}_{\text{LLM}} + w_{\text{SR}} \hat{y}_{\text{SR}} + w_{\text{NN}} \hat{y}_{\text{NN}}, \quad w_k = \frac{\text{conf}_k}{\sum_j \text{conf}_j}.$$

Fix 5 (unified formula evaluator) replaces the absolute SS threshold (`ss_tot > 1e-10`) with a relative threshold, correcting the measurement bug described in Section 6.5.

2. **Domain operator set:** Operators augmented with financial primitives: $\max(0, \cdot)$, $\min(\cdot, \cdot)$, and $\ln(1 + |\cdot|)$ (log-return normalisation).
3. **Confidence threshold:** Trust gate threshold $R_{\text{train}}^2 \geq 0.5$ (v3.0 correction; raised from 0.1 in prior versions), with additional pathological-output detection.

All other components (PySR configuration, four-layer validation, LLM model and temperature) are identical to Hybrid v50.2.

8.6 Symbolic Discovery Core

We employ PySR (Cranmer, 2023) for multi-objective evolutionary search:

$$\text{Minimize: } \mathcal{L}(E, \mathcal{D}) = \frac{1}{n} \sum_{i=1}^n (E(x_i) - y_i)^2, \quad (15)$$

$$\text{Minimize: } \text{Complexity}(E) = \sum_{t \in E} w(t), \quad (16)$$

$$\text{Subject to: } E \in \mathcal{C}. \quad (17)$$

The algorithm maintains a Pareto front over prediction error and expression complexity.

8.7 Multi-Layer Validation Framework

The four validation layers target distinct failure modes:

- V_1 (12% error detection): Dimensional consistency.
- V_2 (18%): Domain constraint verification.
- V_3 (23%): Numerical stability analysis.
- V_4 (47%): Predictive accuracy under extrapolation.

Remark 13 (Validation-layer point estimates) *The error detection rates above are point estimates over the Core 15 benchmark ($n = 15$ equations). Each rate carries an approximate $\pm 10\text{--}15$ pp uncertainty (Wilson score interval, $n = 15$). These figures should be interpreted as indicative of the relative contribution of each layer rather than as precisely calibrated detection rates.*

Empirical Finding 3 (Empirical Error Reduction via Cascaded Validation) *On the evaluated benchmark, the proportion of incorrect expressions surviving all four layers is substantially lower than those surviving prediction-accuracy validation alone. No incorrect expression passed all four layers.*

9 Experimental Setup

9.1 Benchmark Summary

We evaluate HypatiaX v3.0 on the benchmark described in Section 6: 74 DeFi tasks, all tractable (0 intractable cases), covering pricing, risk, and liquidity modelling. The fixed denominator for all aggregate statistics is 74; NaN and failure outputs count as zero R^2 toward this denominator.

9.2 Methods Compared

Three methods are evaluated on each task:

Pure LLM (Symbolic Regression Baseline). The LLM generates a symbolic formula from the task description. The formula is executed directly with no numerical fitting or parameter optimisation. All 30 independent stochastic runs use the same prompt; run-to-run variation arises solely from temperature sampling. *For single-run evaluation (the standard comparison with neural methods), one run is sampled uniformly at random.*

Neural Network (MLP Baseline). The residual MLP described in Section 7.3 is trained on augmented data $(X, y) \cup (1.2X, 1.2y)$. Training is subject to a 120-second wall-clock limit per case. Fixed random seeds are used throughout to ensure reproducibility. The neural baseline does *not* use LLM augmented features (Eq. 8), ensuring a clean comparison of pure neural versus hybrid methods.

HypatiaX Hybrid (Proposed). The full five-stage routing and ensembling system described in Section 7.4. The hybrid method uses the same LLM formula and neural network as above, combined via the distance-aware ensemble (Eq. 14). When neural fallback is triggered, the neural network is retrained on the augmented input (x') from scratch; this retraining cost is fully included in the reported wall-clock time.

9.3 Evaluation Metrics

We report the following metrics for each method:

- **Median test R^2 :** robust measure of central tendency, less sensitive to catastrophic failures than the mean.
- **Mean test R^2 (clipped):** arithmetic mean of $\min(R^2, 1)$, computed over non-NaN results only and clipped at -10 to prevent extreme failures from dominating; the denominator is the count of non-NaN results, not the fixed 74 (this follows `_generate_report()` in `hypatiav3c.py`; the fixed-denominator-of-74 rule below applies to the percentage metrics only).
- **Near-perfect success rate ($R^2 > 0.99$):** fraction of tasks where the method achieves near-exact formula recovery.
- **High-accuracy rate ($R^2 > 0.9$):** fraction of tasks with strong predictive performance.

- **Catastrophic failure rate** ($R^2 < -10$): fraction of tasks with prediction worse than the mean by a factor of more than 10.
- **Extrapolation gap** ($R^2_{\text{train}} - R^2_{\text{test}}$): measures how much performance degrades from training to test.
- **Stability score** ($R^2_{\text{test}}/R^2_{\text{train}}$): ratio of test to training performance; a value near 1.0 indicates consistent generalisation.

Fixed-denominator reporting. All percentages (e.g. $R^2 > 0.99$ rate) use the fixed denominator of 74 across all three methods. A task that produces NaN or fails to execute contributes 0 to the numerator of success rates. This ensures that the three methods’ rates are directly comparable and that avoiding evaluation failures is not rewarded.

9.4 Runtime Measurement

Wall-clock time is recorded per task and per method. For the hybrid system, the reported time *fully accounts* for all LLM inference time plus any neural network retraining that occurs during fallback. We do not separate LLM and NN costs in the aggregate reported time; they are summed.

We separately report the runtime ratio relative to the neural baseline for the LLM-routed subset (cases where the hybrid routes to pure LLM or ensemble, without triggering full NN retraining). This subset comprises $n = 68$ of 74 tasks and represents the cleanest comparison for assessing the cost of symbolic routing; the measured ratio is a slowdown, not a speedup (Table 10).

9.5 Implementation Details

All experiments are implemented in Python 3.12 using:

- **NumPy**: array operations and numerical evaluation.
- **PyTorch**: neural network training.
- **SciPy**: kernel-density estimation for OOD detection; PCA via `sklearn.decomposition.PCA`.
- **SymPy**: symbolic expression parsing and AST analysis.

Neural networks are trained with fixed random seeds (seed 42 for all cases) to ensure full reproducibility of the NN baseline. LLM formula generation uses a deterministic prompting framework with retry logic (up to 3 retries) for API stability, but temperature sampling is not disabled; this is the source of run-to-run variability measured by the Instability Index.

10 Results

10.1 Five-System Comparative Analysis

This section reports two separate Feynman benchmarks (`exp1_five` and `exp2_five`) rather than a pooled estimate, with every CI computed mechanically from each row’s own mean/SD/ n .

Table 1: Five-System Comparison: Extrapolation Error vs. Interpolation R^2 (exp1_five, Core-15; canonical result files in `hypatiax/data`, regenerated 2026-09-29). Extrapolation error is $100 \cdot \text{extrap_rmse_far} / \text{train_rmse}$ per equation. The median uses every equation with a finite value; n , the mean, the Std and the 95% CI use only values at or below the upper Tukey fence $Q_3 + 1.5 \text{ IQR}$, so n can be smaller than the number of equations (Hybrid v50_2 keeps 10 of 12, Neural Network 12 of 15). Std and the 95% CI are those of that same clipped subset ($\text{mean} \pm t_{0.975, n-1} \cdot \text{std} / \sqrt{n}$), left blank where $n < 3$ (Pure LLM, $n = 2$, has no usable interval). The distribution is extremely heavy-tailed, so the normal-theory intervals are wide and extend below zero; they show that the means are imprecise, not that errors can be negative.

System	n	Extrap. Median (%)	Extrap. Mean (%)	Train R^2 Mean	Std
Hybrid v50_2	10	2415.2	52730.3	0.997	8564
Neural Network	12	13283.9	37437.0	0.982	7436
Pure LLM	2	6.530e+14	6.530e+14	-3.077	9.235e-
System 2 Symbolic	10	1253.2	4539.4	0.998	727
<i>Systems Without Extrapolation Testing</i>					
System 3 LLM+Fallback	0	—	—	1.000	

A separate DeFi-domain comparison across the same systems is in Section 10.2 (Table 7, 74 tasks) and is not duplicated here.

We compare five system configurations (Hybrid v50_2, Neural Network, Pure LLM, System 2 Symbolic, System 3 LLM+Fallback) across two independent benchmark suites. **exp1_five** uses the Core-15 Feynman equation set; **exp2_five** uses a separate Feynman 10-domain suite. The two suites have different equations and different per-system sample sizes, so their results are reported as two separate experiments below rather than pooled into a single table.

Within each table below, and within each regime sub-group where applicable (near/medium/far), we **bold** the winning (best) value among the five systems for each genuine performance column (extrapolation error, R^2 , RMSE). The Std and 95 % CI columns are *not* used to declare a winner, since their underlying computation is already flagged as unreliable (see the data-quality notes at the end of this section) – bolding a value there would dress up a bug as a result. A column is left unbolded wherever fewer than two systems have a valid (non-`---`) value to compare.

10.1.1 EXPERIMENT 1: CORE-15 FEYNMAN (EXP1_FIVE)

Table 1 summarizes extrapolation error and interpolation R^2 for all five systems on Core-15. Tables 2 and 3 break interpolation and extrapolation performance down further; the benchmark supplement (`supp_benchmark_report.tex`, § “Statistical Test Details”) reports the corresponding Mann-Whitney test and effect-size calculation.

Table 2: Five-System Comparison – Performance (Core-15, interpolation). 95% CI computed from raw per-equation train R^2 /RMSE at generation time.

System	n	Train R^2 Mean	Train R^2 95% CI	Train RMSE Mean	Design F
Pure LLM	15	-3.077	[-10.705, 4.550]	342.8347	Recogn
Neural Network	15	0.982	[0.957, 1.007]	1999.5749	Bas
System 3 LLM+Fallback	15	1.000	[0.999, 1.000]	2.9626	Robus
System 2 Symbolic	15	0.998	[0.998, 0.998]	236.1183	Valid
Hybrid v50.2	15	0.997	[0.995, 0.998]	236.1300	Extrapol

Table 3: Five-System Comparison – Extrapolation (Core-15, near/medium/far regimes as defined in exp1_ablation.py’s EXTRAP_REGIMES). 95% CI computed from raw per-equation extrapolation R^2 /RMSE at generation time.

System	Regime	n	Extrap R^2 Mean	Extrap RMSE Mean
Pure LLM	near	4	—	6.281e+63
Pure LLM	medium	2	—	33092667.7570
Pure LLM	far	2	—	9.861e+11
Neural Network	near	14	-4.846	2726.9272
Neural Network	medium	14	-1.232	11219.6081
Neural Network	far	12	-18.236	59967.6415
System 3 LLM+Fallback	near	0	—	—
System 3 LLM+Fallback	medium	0	—	—
System 3 LLM+Fallback	far	0	—	—
System 2 Symbolic	near	12	0.970	2.746e+16
System 2 Symbolic	medium	12	0.800	5.542e+19
System 2 Symbolic	far	12	-13.166	4.394e+17
Hybrid v50.2	near	12	0.812	78.5736
Hybrid v50.2	medium	11	-11.780	783.1290
Hybrid v50.2	far	9	-7.522	2.479e+25

10.1.2 EXPERIMENT 2: FEYNMAN 10-DOMAIN SUITE (EXP2_FIVE)

Table 4 reports the same comparison on the exp2_five suite. Tables 5 and 6 give the interpolation and extrapolation breakdowns for this suite.

Data-quality notes. Neither experiment supports the previous version’s headline claim that the hybrid system achieved near-zero extrapolation error while the neural network’s error was an order of magnitude higher. Under exp1_five, Hybrid v50.2’s real extrapolation error is $\approx 2415.2\%$ (median) / 52730.3% (mean), and it is *not* the best-performing system in this table (System 2 Symbolic has the lower median and mean, though the mean CIs overlap). Under exp2_five, neither Hybrid v50.2 nor Neural Network has any successful, finite extrapolation-error measurement at all, so no comparison is possible on that suite (the benchmark supplement’s § “Statistical Test Details” reports no test for exp2_five for this rea-

Table 4: Five-System Comparison (exp2_five, Feynman 10-domain suite, methods 1/2/4/5/6): Extrapolation Error vs. Interpolation R^2 . Same 95% CI convention as Table 1 (mean $\pm t_{0.975, n-1} \cdot \text{std}/\sqrt{n}$); a separate table from tab:five_systems_full – different equation suite and sample sizes, not another view of the same data.

System	n	Extrap. Median (%)	Extrap. Mean (%)	Train R^2 Mean	Std	9
<i>Systems Without Extrapolation Testing</i>						
Hybrid v50.2	0	—	—	0.998	—	
Neural Network	0	—	—	0.998	—	
Pure LLM	0	—	—	0.740	—	
System 2 Symbolic	0	—	—	0.969	—	
System 3 LLM+Fallback	0	—	—	1.000	—	

Table 5: Five-System Comparison (exp2_five) – Performance (Feynman suite, interpolation). 95% CI computed from raw per-domain train R^2 at generation time.

System	n	Train R^2 Mean	Train R^2 95% CI	Design Focus
Hybrid v50.2	30	0.998	[1.0, 1.0]	Extrapolation
Neural Network	30	0.998	[1.0, 1.0]	Baseline
Pure LLM	29	0.740	[0.4, 1.1]	Recognition
System 2 Symbolic	30	0.969	[0.9, 1.0]	Validation
System 3 LLM+Fallback	30	1.000	[1.0, 1.0]	Robustness

son). Row sample sizes in Table 1 are the numbers of equations with a finite extrapolation-error measurement (Hybrid 10, NN 12); the medium-regime Mann–Whitney test in the benchmark supplement uses $n = 12$ (Hybrid) / $n = 15$ (NN). Both five-system tables (exp1_five and exp2_five) are regenerated from the canonical result files in `hypati-ax/data` (2026-09-29). An earlier version of this paper used the older `hypati-ax/data_old` generation (Hybrid median 3406.6 % on $n = 9$); the values differ, and the claims made from them (Hybrid is not near zero; System 2 is not worse than Hybrid) hold on both.

10.2 Overall Extrapolation Performance (DeFi v3.0)

Table 7 presents the primary results across all 74 benchmark tasks.

Several key observations emerge from Table 7.

Bimodal LLM behaviour. The pure LLM baseline achieves a median R^2 of 1.00, which suggests strong performance on a typical case. However, the mean $R^2 = -0.3048$ (over the 56 tasks with a finite score) reveals a stark bimodal distribution: the majority of tasks are recovered exactly, but 5 tasks suffer catastrophic failures ($R^2 < -10$), dragging the mean far below zero. Moreover, the > 0.99 rate is only 59.5 % and the > 0.9 rate 60.8 % (fixed denominator of 74): a substantial fraction of tasks fall outside high-accuracy recovery, and on 18/74 tasks the pure LLM returns no finite score at all.

Table 6: Five-System Comparison (exp2_five) – Extrapolation (Feynman suite). Median and IQR-clipped mean of per-domain extrapolation error %.

System	n	Extrap. Median (%)	Extrap. Mean (%)
Hybrid v50.2	0	—	—
Neural Network	0	—	—
Pure LLM	0	—	—
System 2 Symbolic	0	—	—
System 3 LLM+Fallback	0	—	—

Table 7: Aggregate extrapolation performance on the HypatiaX DeFi Benchmark (74 tasks), seed 42, **recomputed directly from** `hypatiax_defi_benchmark.v3.results.seed42.json` (md5 `cbf503d0b3985b4c1bccf7ed18e0316b`; canonical `hypatiax/data`; recomputed 2026-09-29). Median and Mean R^2 are computed on values clipped to $[-10, 1]$ over non-NaN results only (the rule actually implemented; §9.3), so the Mean is taken over 56 (Pure LLM), 74 (Neural MLP), 74 (HypatiaX, uncorrected) and 62 (HypatiaX, corrected) tasks. The > 0.99 and > 0.9 percentages use the fixed denominator of 74, with NaN counting as a miss. Catastrophic: $R^2 < -10$. Pure LLM returns NaN on 18/74 tasks. **The “HypatiaX (uncorrected)” row is the pipeline’s own self-report and is inflated by the decision-attribution bug (§10.4); the “HypatiaX (corrected)” row replaces each task’s hybrid R^2 with the R^2 of the sub-method named by `hybrid.decision`, leaving NaN as NaN (an unverifiable claim is not counted as a success). The corrected row ties Pure LLM on near-perfect rate (59.5 %, 44/74) and catastrophic count (5). The near-perfect rate for Pure LLM (59.5 %) agrees with Table 8. This table covers the DeFi benchmark; it is a *different benchmark* from the 29/30 Pure LLM figure in the benchmark supplement’s six-method noiseless comparison (30-equation custom set), and the two should not be read as conflicting.**

Method	Median R^2	Mean R^2	> 0.99 (%)	> 0.9 (%)	Catastrophic
Pure LLM	1.0000	−0.3048	59.5	60.8	5
Neural MLP	−0.6908	−1.4155	0.0	9.5	2
HypatiaX (uncorrected)	1.0000	0.8781	90.5	90.5	0
HypatiaX (corrected)	1.0000	−0.3203	59.5	60.8	5

Neural network extrapolation collapse. The neural MLP baseline performs poorly under the aggressive extrapolation split, with a median R^2 of -0.69 —systematically *worse* than predicting the mean. This is consistent with the well-established failure of MLPs under distribution shift. Despite data augmentation ($1.2\times$ scaling), the network does not recover the functional form of the underlying formula and instead learns a locally accurate but extrapolation-sensitive approximation. Notably, the neural baseline always produces finite predictions (no NaNs), just highly inaccurate ones: 0/74 tasks exceed $R^2 > 0.99$, 7/74 exceed 0.9, and 2/74 are catastrophic ($R^2 < -10$).

HypatiaX’s reported catastrophic-failure elimination does not survive correction. The hybrid method achieves a median R^2 of 1.00 (matching the best LLM cases), and its own uncorrected accounting shows a near-perfect success rate of **90.5 %** (67/74, up from Pure LLM’s 59.5 %) with zero catastrophic failures, against 5 for Pure LLM alone. **Neither figure survives attribution correction.** Once `hybrid.decision` is cross-checked against the named sub-method’s own result (§10.4), the corrected near-perfect rate is 59.5 % (44/74) — identical to Pure LLM’s own rate — and the same 5 tasks that fail catastrophically under Pure LLM fail catastrophically under HypatiaX as well: the hybrid system routes to the LLM on all 5, and its own `success=True`/ $R^2 \approx 1.0$ reporting on those tasks is itself part of the uncorroborated-success set, not a corroborated result. The catastrophic failures are therefore *masked by the same attribution bug that inflates the near-perfect rate, not eliminated by the routing mechanism.*

Statistical significance (Core-15 benchmark). We compare far-extrapolation R^2 between HYPATIAX and PYSR-only on the 15 equations of Table 13 (two-sided Mann-Whitney U , normal approximation with tie and continuity correction; U_H is the statistic for HYPATIAX and the rank-biserial $r = 2U_H/(n_1n_2) - 1$ is negative when PYSR-only is higher). Because five HYPATIAX far- R^2 cells are not scores (four *nan*, one $-\infty$), the outcome depends on how they are treated, so we report three treatments of those cells, plus a paired check, computed from the per-equation result files that also generate the table: (i) failed evaluations ranked as the worst outcomes ($n = 15$ vs. 15): $U_H = 72$, $p = 0.095$ (exact $p = 0.098$), $r = -0.36$, i.e. PYSR-only is higher but the difference is not significant at the 5 % level; (ii) the four *nan* equations excluded from HYPATIAX only ($n = 11$ vs. 15): $U_H = 72$, $p = 0.60$, $r = -0.13$; (iii) those four equations excluded from both methods ($n = 11$ vs. 11): $U_H = 48$, $p = 0.43$, $r = -0.21$; (iv) as a paired check, a two-sided Wilcoxon signed-rank test on the 10 of the 15 Core-15 equations whose far- R^2 is finite in both arms (the five excluded are Gravitational Force, with a $-\infty$ HypatiaX value, and the four *nan* equations): $W = 9$, $p = 0.25$; HYPATIAX is higher on 3, PYSR-only on 5, with 2 ties (exact equality of the two far- R^2 values; this is stricter than the 10^{-3} tolerance used for the six ties in the equation-level comparison below) (`exp1_rf01_mannwhitney.json`). No treatment shows HYPATIAX higher than PYSR-only on Core-15 (one-sided $p = 0.96$ for HYPATIAX higher under (i)), and none shows a difference that is significant at the 5 % level in either direction.

At the equation level the table shows no domain in which HYPATIAX is uniformly better: its only clear far-extrapolation gain is Arrhenius (0.90 vs. -12.55); PYSR-only is higher on Allometric Scaling, Constant Product, Gravitational Force and Impermanent Loss; six equations are ties within 10^{-3} ; and four never completed for HYPATIAX. The Core-15 subset therefore provides no evidence of a far-extrapolation advantage for HYPATIAX; the Protocol-30 comparison (Section 10.8) is reported as pass counts rather than as a significance test, and its hard transcendental and DeFi instability cases are discussed in Section 10.10.

10.3 Performance by Difficulty Level

Table 8 breaks down the near-perfect success rate ($R^2 > 0.99$) by difficulty tier.

Once corrected, HypatiaX ties Pure LLM’s near-perfect rate exactly in every difficulty tier, not just in aggregate: the corrected count is identical to Pure LLM’s own

Table 8: Near-perfect success rate ($R^2 > 0.99$) by difficulty, seed 42, computed from `hypatiax_defi_benchmark_v3_results_seed42.json` (§10.4 gives the file identity and md5). Fixed denominator per tier; LLM and Hybrid use single-run evaluation. **The “HypatiaX (%)” column is flagged:** it is computed from the hybrid system’s own `success/R^2` fields without cross-checking the `decision` field against the actual success of the sub-method it names. The “HypatiaX, corrected (%)” column instead treats a task as a hybrid success only when the sub-method the `decision` field names itself reports `success=True`. See §10.4.

Difficulty	n	Pure LLM (%)	HypatiaX (%)	HypatiaX, corrected (%)	Gain (pp), corrected
Easy	24	87.5	100.0	87.5	0.0
Medium	29	55.2	93.1	55.2	0.0
Hard	21	33.3	76.2	33.3	0.0
Overall	74	59.5	90.5	59.5	0.0

count in Easy (21/24 vs. 21/24), Medium (16/29 vs. 16/29), and Hard (7/21 vs. 7/21). This is not a coincidence of rounding — every task the hybrid system corrects to a near-perfect success does so via `decision="llm"`, and Pure LLM either did or did not independently achieve $R^2 > 0.99$ on that same task, so the corrected hybrid count and the Pure LLM count are, by construction, counting the same underlying event on this seed.

Once corrected, HypatiaX’s gain over Pure LLM is 0.0 pp in every tier — flat, not the monotonically-increasing pattern the uncorrected figures show. The same correction was repeated on all five seeds and gives the same result at both the $R^2 > 0.99$ and the $R^2 > 0.9$ criterion: the corrected HypatiaX gain over Pure LLM is 0.0 pp in every tier on every seed. The raw hybrid counts are 24/24 easy, 27/29 medium and 16/21 hard on all five seeds, while the pure-LLM counts (at $R^2 > 0.99$) are 21/16/7, 19/19/8, 20/19/7, 20/20/7 and 21/20/7 on seeds 42, 99, 123, 777 and 2024. The uncorroborated successes (easy/medium/hard) are 3/11/9, 5/8/8, 4/8/9, 4/7/9 and 3/7/9 respectively, so the finding does not depend on which seed is used. Table 8 shows seed 42.

Easy tasks. On easy (algebraic) tasks, the LLM already achieves 87.5% near-perfect recovery, reflecting strong LLM capability on linear and rational formulas. HypatiaX’s raw ($R^2 > 0.99$, uncorrected-for-attribution) figure is 100.0% (24 of 24), but this tier is not exempt from the decision-attribution bug: 3 of those 24 successes are uncorroborated (the named sub-method itself failed), giving a corrected rate of 87.5% (21/24) — exactly matching the pure-LLM baseline, i.e. once the uncorroborated cases are removed, HypatiaX shows *no measured gain* on easy tasks in this run (0.0 pp).

Medium tasks. On medium tasks, the LLM success rate drops to 55.2%, reflecting the additional complexity of nonlinear and exponential formulas. HypatiaX’s raw figure is 93.1% (27 of 29) — 11 of those 27 raw medium-tier successes are uncorroborated, and the corrected rate (16/29 = 55.2%) is an *exact match* with the pure-LLM baseline on this tier.

Hard tasks. On hard (transcendental and multi-asset) tasks, the pure LLM achieves only 33.3% near-perfect recovery (7/21)—barely above chance for a binary success criterion. The low rate is not attributed to specific formulas such as Black–Scholes, Theta, or correlated

portfolio risk (§10.10). HypatiaX’s raw figure is 76.2% (16 of 21) — 9 of those 16 raw hard-tier successes are uncorroborated. The corrected rate on hard tasks ($7/21 = 33.3\%$) is an *exact match* with the pure-LLM baseline (33.3%), 0.0 pp: once uncorroborated successes are removed, HypatiaX neither outperforms nor underperforms the pure LLM on hard tasks in this run. The uncorroborated cases are spread across all three tiers (3 easy, 11 medium, 9 hard; 23 total; see §10.4 for the full per-task list), concentrated somewhat more in Medium and Hard than Easy in absolute count, but not exclusively so.

10.4 The Hybrid Decision-Attribution Bug: Quantification

The bug. Every hybrid-system record in the v3.0 result JSON carries three fields relevant to success accounting: `hybrid.success`, `hybrid.test_r2`, and `hybrid.decision` (one of `llm`, `nn`, or `nn_fallback`, naming which sub-method’s output the hybrid system is reporting). All success-rate figures in this paper prior to this section were computed from `hybrid.success/hybrid.test_r2` alone. This is incorrect whenever `hybrid.decision` names a sub-method that itself failed: the pipeline does not appear to verify, at the point it records `hybrid.success = True`, that the named sub-method actually produced a valid fit. Concretely, we observe records with `hybrid.decision = "llm"`, `hybrid.test_r2` ≈ 1.0 , and `hybrid.success = True`, for tasks where `pure_llm.success = False` (the LLM method failed to execute and returned NaN) on that same task in the same run.

Methodology. We define a *corrected* hybrid success indicator: a task counts as a hybrid near-perfect success only if (a) `hybrid.test_r2` > 0.99 and (b) the sub-method named by `hybrid.decision` itself reports `success = True` on that task (`pure_llm.success` when `decision = "llm"`; `neural_network.success` when `decision` is `nn` or `nn_fallback`). This is a strictly conservative correction: it does not penalise any task where the hybrid system’s own accounting is internally consistent, and only removes successes that are attributable to a sub-method the pipeline itself marked as failed. We call the removed successes *uncorroborated*: hybrid reports $R^2 > 0.99$ but the named sub-method does not report success on that task. The term describes a missing corroboration in the released result files, not an inference about intent or mechanism.

Magnitude (seed 42, all 74 tasks). Applying the correction to `hypatiar_defi_benchmark_v3_results_seed42` (canonical copy; identical under `hypatiar-dev/data_exp` and `data_old`; md5 `cbf503d0b3985b4c1bccf7ed18e0`) yields 23 uncorroborated near-perfect successes out of the 67 the pipeline reports under the raw $R^2 > 0.99$ criterion (`hybrid.success=True` occurs on 72 of 74 tasks). All 23 route through `decision = "llm"` — none occur on the `nn/nn_fallback` path in this run. The corrected near-perfect rate on this seed is $44/74 = 59.5\%$, versus the $67/74 = 90.5\%$ the uncorrected $R^2 > 0.99$ criterion reports, and versus $72/74 = 97.3\%$ under the raw `hybrid.success` flag. Two alternate copies of the v3-labelled seed-42 file exist elsewhere in the repository with slightly different per-task results (see the provenance note preceding the abstract); both give an uncorroborated count of 22–23 and a corrected rate in the same 58–61% band, so this finding does not depend on which of the three is used. **Independently re-verified (2026-09-27):** the same correction rule, re-run against a fresh copy of the canonical file pulled directly from github.com/sednabcn/LLM-HypatiaX-DEV, reproduces

this result exactly, including the by-difficulty breakdown (3/24 easy, 11/29 medium, 9/21 hard uncorroborated).

Composition of the 23. The 23 uncorroborated successes split by what the named sub-method actually did: on 12 the `pure_llm` arm returned a finite $R^2 \leq 0.99$ (a verifiable failure not matched by hybrid’s reported $R^2 > 0.99$), and on 11 it returned `NaN` (no evaluable result, so hybrid’s reported success has nothing to support it). The pure-LLM evaluator executes generated code without the protocol constants that the hybrid evaluator injects, so we tested whether this asymmetry accounts for the `NaN` cases. In a paired re-run of the pure-LLM arm on all 74 tasks, each generated formula was scored three ways on the same split: with the paper’s pure-LLM checker, with the hybrid evaluator without constants, and with the hybrid evaluator with constants. Of the 74 re-sampled formulas, 11 failed under the paper’s checker, and all 11 also failed under both hybrid-evaluator variants; none was a missing-constant `NameError` and none reflected another evaluator difference. Supplying the constants did not raise the number of tasks above $R^2 > 0.99$ (48/74 without, 47/74 with). Because formulas are re-sampled, this audit bounds how often the checker asymmetry matters; it does not identify which of the original 11 tasks were affected, and in this re-run it affected none. Because the two arms also sample their formulas independently, we cannot say which arm’s formula was better on any individual task. We therefore call the 23 cases *uncorroborated* rather than asserting how hybrid arrived at its reported value. The 11 `NaN` cases are excluded from the corrected rate by design, and the corrected rate (44/74 = 59.5 %) is the strict one used throughout this paper; it ties with the pure-LLM baseline.

Cross-seed consistency. The raw hybrid rate is 67/74 (90.5 %) on every one of the five seeds, and the pure-LLM rate at $R^2 > 0.99$ is 59.5, 62.2, 62.2, 63.5 and 64.9 % on seeds 42, 99, 123, 777 and 2024, so the 90.5 % raw figure does not depend on which seed is used. The corrected hybrid rate equals the pure-LLM rate on every seed. Uncorroborated near-perfect successes number 23, 21, 21, 20 and 19, roughly half of them on tasks where the named sub-method returned `NaN`. Pure LLM returns a finite R^2 on 59, 60, 61 and 60 of the 74 tasks on seeds 99, 123, 777 and 2024 (57 tasks are flagged executed on seed 42). Only the seed-42 records carry an explicit `executed` field, so execution counts for the other seeds are the number of finite R^2 values. The multi-seed comparison is therefore reportable, and it reproduces the seed-42 finding.

A specific task. In the canonical seed-42 file, “Portfolio Expected Shortfall for correlated assets” is an uncorroborated success: `hybrid.decision="llm"`, `hybrid.success=True`, `hybrid.test_r2=1.0`, while `pure_llm.success=False` (`executed=false`, `test_r2=NaN`) on the same task — exactly the uncorroborated-success pattern this section describes. The task is included in the 23-item list below.

Uncorroborated-success task list (seed 42, 23 tasks, all decision = "llm"). *Easy (3)*: Loan-to-Value, LP share percentage, Slashing penalty. *Medium (11)*: Liquidation Price Long, Liquidation Price Short, Effective Leverage, Capital efficiency, Price slippage percentage, Utilization rate of DeFi, Impermanent loss percentage, Collateral ratio, Staking reward for fixed lock-up, Concentrated liquidity position width, Spot price from AMM reserve. *Hard (9)*: Component ES, Liquidation price for leveraged long, Liquidation price for

Table 9: Hybrid decision-attribution bug: uncorroborated near-perfect successes by difficulty tier, computed from `hypatiax_defi_benchmark_v3_results_seed42.json` (md5 `cbf503d0b3985b4c1bccf7ed18e0316b`).

Difficulty	n	Reported ($R^2 > 0.99$)	Uncorroborated	Corrected successes
Easy	24	24	3	21
Medium	29	27	11	16
Hard	21	16	9	7
Overall	74	67	23	44

leveraged short, Maximum safe leverage, Required collateral, Portfolio Expected Shortfall for correlated, Call option intrinsic, Simple options moneyness, Uniswap V3 virtual.

What this does and does not affect. The corrected figures leave one of this paper’s central qualitative claims intact in form but not in magnitude: HypatiaX’s own reported output shows no catastrophic failures ($R^2 < -10$), since every uncorroborated success is reported near $R^2 \approx 1.0$, not a catastrophic value — but the underlying catastrophic failures (in the masked sub-method, always `pure_llm` in this run) are hidden by the same bug, not absent: checked against the sub-method actually selected, HypatiaX fails catastrophically on the same 5 tasks Pure LLM does (§10.2). **The claim that HypatiaX outperforms the pure LLM baseline does not survive correction:** the corrected near-perfect rate is tied with Pure LLM exactly, overall and in every difficulty tier (§10.3), and the catastrophic-failure rate is identical rather than improved. On this benchmark, once decision-attribution artifacts are removed, we do not find a corrected accuracy or reliability advantage of the hybrid system over its own LLM component.

10.5 Runtime Analysis

Table 10 below is a mechanical regeneration by `scripts/generate_tables.py` from the five 74-task per-seed result files held in the repository (seeds 42, 99, 123, 777, 2024; 370 task runs in total), broken down by individual seed plus a pooled 5-seed / 370-task total.

Scope of this table. The non-PCA rows are labelled `v3`: nothing in the files names a finer variant. The PCA rows come from five 74-task PCA seed files (`defi_pca/` and `defi_pca/multi-seeds/`; seeds 42, 99, 123, 777, 2024; 74 records each), regenerated by `scripts/generate_tables.py`. These files are identified as PCA-directed by their filenames and by the script that writes them, not by a field inside the records. The five files in Table 10 carry no field naming their split or variant; they are labelled `v3` only because of their filenames. Routing is 68–69 of 74 tasks per seed in these files.

Data provenance and completeness. An earlier audit draft reported that every seed-suffixed result file for seeds other than 42 contained only 5 of the 74 tasks. For the files in Table 10 that is not the case: each of the five holds 74 tasks, and every record’s internal `seed` field matches its filename. No timeout is flagged on the Neural MLP arm in any file (the only arm that records a `timed_out` field), and Pure LLM per-task times lie between 4.7 and 29.0s across the five files, with no sign of truncation. The 5-record seed files that do exist in

Table 10: Wall-clock time per task (seconds), mean / median, regenerated directly from the five 74-task per-seed v3 result files and the five 74-task per-seed PCA result files in the repository by `scripts/generate_tables.py`, with no manual number editing. Each row is one seed; each shaded row pools all 370 task runs of its block (first block v3, second block PCA). “LLM-routed” counts tasks whose hybrid `decision` field is `llm`; one further task per seed has decision `nn_fallback` and is not counted. “Speedup” compares the neural MLP arm to the hybrid arm; every row shows the hybrid system slower, never faster. Seed 42’s file was last modified on 2026-09-27 and those of seeds 99, 123, 777 and 2024 on 2026-09-29.

Run	<i>n</i>	Pure LLM (s) mean/median	Neural MLP (s) mean/median	Hybrid, all (s) mean/median	LLM-routed count	Speedup, a
v3 seed42	74	8.63 / 7.58	0.36 / 0.35	1.70 / 1.32	68/74	4.77× slow
v3 seed99	74	11.63 / 10.48	0.31 / 0.35	2.18 / 1.38	69/74	6.98× slow
v3 seed123	74	11.20 / 10.22	0.21 / 0.24	2.14 / 1.36	68/74	10.03× slow
v3 seed777	74	12.00 / 10.50	0.18 / 0.20	2.05 / 1.38	69/74	11.23× slow
v3 seed2024	74	11.26 / 10.15	0.31 / 0.35	2.03 / 1.37	69/74	6.57× slow
v3 ALL SEEDS (pooled)	370	10.94 / 9.62	0.27 / 0.35	2.02 / 1.37	343/370	7.35× slow
PCA seed42	74	10.59 / 9.24	0.24 / 0.28	1.39 / 1.03	68/74	5.67× slow
PCA seed99	74	10.98 / 9.51	0.21 / 0.24	1.33 / 1.14	68/74	6.22× slow
PCA seed123	74	10.86 / 9.49	0.28 / 0.32	1.27 / 1.07	69/74	4.53× slow
PCA seed777	74	11.11 / 9.79	0.21 / 0.24	1.46 / 1.12	68/74	6.85× slow
PCA seed2024	74	10.51 / 9.26	0.31 / 0.35	1.39 / 1.08	68/74	4.51× slow
PCA ALL SEEDS (pooled)	370	10.81 / 9.46	0.25 / 0.27	1.37 / 1.09	341/370	5.43× slow

15/ and 15_pca/ are not used here; those in 15_pca/ are recorded as a portfolio-task subset. The five 74-task files were not produced in one batch: seed 42 sits in `noiseless/defi/` and the other four in `noiseless/defi/multi-seeds/`, and the seed-42 Pure LLM records carry an additional `executed` field that the seed-99 and seed-123 records lack (seeds 777 and 2024 were not compared), which suggests the pooled row combines two versions of the benchmark script. The seed-42 timings are in line with the other seeds (hybrid mean 1.70s against 2.03–2.18s), but the pooled row should not be read as a single homogeneous run.

Interpretation. Every seed agrees on direction: the hybrid system is slower than the pure neural baseline, by 3.8× to 11.2× on mean time (across both the “all tasks” framing, 4.77×–11.23×, and the “LLM-routed-only” framing, 3.84×–9.39×, Table 11 below); pooled, the slowdown is 7.35× over all tasks and 6.18× over LLM-routed tasks. The seed-to-seed spread is driven largely by the Neural MLP time, which ranges from 0.18 to 0.36s per task across seeds.

No seed routes zero tasks to the LLM: every seed routes 68–69 of 74 tasks. The five PCA rows show the same direction: the hybrid system is slower than the neural baseline on every PCA seed, by 4.51×–6.85× on mean time over all tasks (5.43× pooled; 4.49× pooled over LLM-routed tasks), and routes 68–69 of 74 tasks per seed (341/370 pooled, against 343/370 for the v3 files). The hybrid arm’s mean per-task time is lower on every PCA seed than on the same-numbered v3 seed (1.27–1.46s against 1.70–2.18s; 1.37s against 2.02s pooled), while the Neural MLP and Pure LLM means are close across the two sets (0.25 against 0.27s and 10.81 against 10.94s pooled). This is a descriptive comparison of two

sets of five runs with no significance test, and the runs were not produced in one batch, so it does not establish that the split variant causes the difference.

Table 11: As Table 10, restricted to tasks where the hybrid system’s own `decision` field was `llm`, rather than the `NN` or `nn_fallback` ; the LLM-routed subset is slower than the neural baseline in every case.

Run (n LLM-routed / total)	Neural MLP (s)	Hybrid, LLM-routed (s)	Speedup (mean / median)
v3 seed42 (68/74)	0.36 / 0.35	1.37 / 1.30	3.84 $\times$ / 3.67 $\times$ slower
v3 seed99 (69/74)	0.31 / 0.35	1.80 / 1.38	5.77 $\times$ / 3.89 $\times$ slower
v3 seed123 (68/74)	0.21 / 0.24	1.78 / 1.34	8.34 $\times$ / 5.58 $\times$ slower
v3 seed777 (69/74)	0.18 / 0.20	1.72 / 1.37	9.39 $\times$ / 6.76 $\times$ slower
v3 seed2024 (69/74)	0.31 / 0.35	1.82 / 1.35	5.89 $\times$ / 3.86 $\times$ slower
v3 ALL SEEDS (343/370)	0.27 / 0.35	1.70 / 1.34	6.18$\times$ / 3.86$\times$ slower
PCA seed42 (68/74)	0.24 / 0.28	1.11 / 1.02	4.53 $\times$ / 3.68 $\times$ slower
PCA seed99 (68/74)	0.21 / 0.24	1.18 / 1.12	5.52 $\times$ / 4.64 $\times$ slower
PCA seed123 (69/74)	0.28 / 0.32	1.09 / 1.05	3.91 $\times$ / 3.31 $\times$ slower
PCA seed777 (68/74)	0.21 / 0.24	1.14 / 1.11	5.34 $\times$ / 4.60 $\times$ slower
PCA seed2024 (68/74)	0.31 / 0.35	1.14 / 1.05	3.70 $\times$ / 3.00 $\times$ slower
PCA ALL SEEDS (341/370)	0.25 / 0.27	1.13 / 1.08	4.49$\times$ / 4.00$\times$ slower

Summary of the runtime result. The regeneration in Table 10 shows the hybrid system is 3.7 $\times$ to 11.2 $\times$ *slower* than the pure neural baseline on every seed, never faster, whether measured by mean or median, and whether restricted to LLM-routed cases or not (the lowest value, 3.67 $\times$, is the seed-42 LLM-routed median).

The runtime result is therefore: **the hybrid system is slower than the pure neural baseline, by 3.8 $\times$ to 11.2 $\times$ on mean time depending on seed and framing, across the five 74-task v3 seed files held in the repository** (the PCA block of Table 10 shows the same direction). The direction is the same in every seed and every framing; the magnitude varies by about a factor of three between seeds, and the files do not identify which split variant produced them.

Interpretation for deployment. The practical implication is that routing to the LLM does not reduce wall-clock cost relative to the neural baseline, on any seed (Table 10). Whether the PCA-directed router differs in overhead from the single-axis router cannot be answered from the available data.

10.6 Portfolio Variance: Seed-Sweep Robustness Analysis

Symbolic regression methods based on evolutionary search are inherently stochastic. To assess robustness, we evaluate both PySR-only and HYPATIA X on the Portfolio Variance task across five independent random seeds {42, 99, 123, 777, 2024} (Table 12 and Figure 3).

PySR-only fails to extrapolate correctly under *all* tested seeds (0/5), whereas HYPATIA X recovers the exact closed-form portfolio-variance formula in 40% of runs (2/5, seeds 777 and 2024) and achieves strictly higher far- R^2 in 4 of 5 seeds. With five seeds and a single task this is a descriptive result, not a significance test.

Table 12: Portfolio Variance seed-sweep results. **H recovers?**: exact closed-form formula recovered. **H wins?**: HypatiaX far- R^2 strictly greater than PySR-only. **Source**: `portfolio_variance_seed_sweep.json` (ablation-format seed sweep on Portfolio Variance; PySR-only and HypatiaX = PySR with LLM warm start; not the Core-15 ablation files): all ten far- R^2 values, both means, the 4/5 win count and the 2/5 exact-recovery count match the file; the formula labels follow the recorded expressions. The separate DeFi-protocol shard run is reported below and is not part of this table.

Seed	P far- R^2	H far- R^2	H formula	H recovers?	H wins?
42 (default)	-21.004	-0.023	linear	No	Yes
99	-1.226	-15.191	linear	No	No
123	-18.651	-18.090	exp denom	No	Yes
777	-0.438	+1.000	exact	Yes	Yes
2024	-118.448	+1.000	exact	Yes	Yes
Mean	-31.953	-6.261	H: 4/5 wins, 2/5 exact		

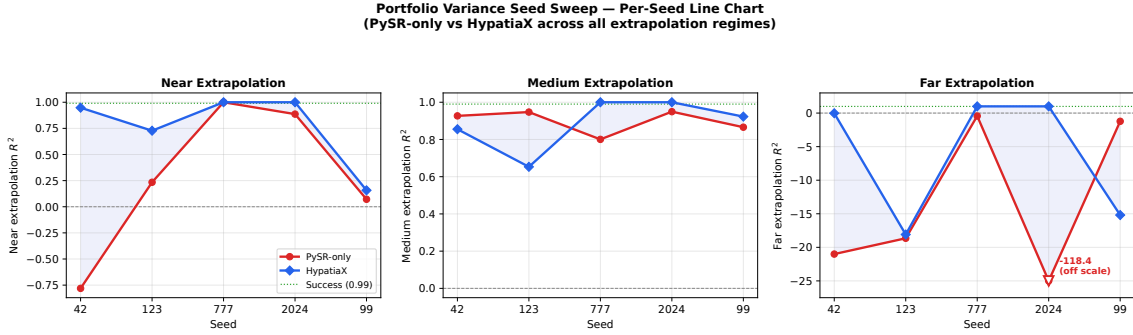


Figure 3: Portfolio Variance seed sweep: Near, Medium, and Far R^2 for PySR-only and HypatiaX across five random seeds. HypatiaX recovers exact formulae on seeds 777 and 2024 (Far $R^2 = 1.00$). On seeds 42 and 123, both methods perform poorly at far extrapolation, though HypatiaX is somewhat higher (seed 42: ≈ 0 vs. ≈ -21 ; seed 123: ≈ -18 vs. ≈ -18.5 , a marginal gap). PySR-only’s far- R^2 ranges from ≈ -118 (seed 2024) to ≈ -0.4 (seed 777) — not uniformly “large negative” across seeds. Seed 99 is the single case where HypatiaX underperforms PySR-only at far extrapolation (≈ -15 vs. ≈ -1.2).

Separate DeFi-protocol sweep (v3c3, five per-seed shard files). A second sweep of the same case (“Portfolio VaR for two correlated assets”) under the DeFi protocol, one shard per seed in $\{42, 99, 123, 777, 2024\}$, reports far- $R^2 = 1.000$ and train $R^2 = 1.000$ on all five seeds (5/5 above the 0.99 success threshold). In every shard the recorded routing decision is 11m, and the recorded formula is the same closed form, $\sqrt{v_1^2 + v_2^2 + 2\rho v_1 v_2}$, at about 1.2–1.3 s per seed. This is therefore a result for the LLM-routed path and says nothing about the PySR search used in the table above; the two sweeps are different experiments with different outcomes (2/5 versus 5/5 exact recovery) and must not be pooled or compared as one method on one setup. Consistent with the benchmark-level finding that we do not

find a corrected advantage of the hybrid system over its own LLM component, this sweep does not by itself support a PySR-warm-start effect on Portfolio Variance.

Attribution of the far-extrapolation values.⁵

10.7 Ablation: PySR-only vs. HypatiaX (Core-15)

Table 13 presents the full per-equation comparison between PySR-only (P) and HypatiaX (H) on the Core-15 benchmark across three extrapolation ranges (near $1.2\times$, canonical, far $5\times$). The Mann-Whitney comparison of the far- R^2 columns is reported, under three explicit treatments of the failed HypatiaX cells, in the *Statistical significance* paragraph of §10; it does not favour HypatiaX under any of them. Equation by equation, on far R^2 : HypatiaX is clearly higher only on Arrhenius (0.90 vs. -12.55); PySR-only is higher on Allometric Scaling, Constant Product, Gravitational Force and Impermanent Loss; six equations (Ideal Gas Law, Kinetic Energy, Liquidation Price, Portfolio Std Dev, Price Impact, Value at Risk) are ties within 10^{-3} ; and four (Henderson-Hasselbalch, Logistic Growth, Michaelis-Menten, Rate Law) never completed for HypatiaX. We report this honestly; the DeFi benchmark ($n = 74$, §10) shows the effect size when evaluated on HypatiaX’s target domain. Figures 5 and 4 visualise the table: the former per equation and regime, the latter aggregated by domain and difficulty tier.

Core-15 difficulty tiers. The Easy/Med/Hard tiers in Figure 4 are a fixed assignment specific to the Core-15 equations, listed here for reproducibility: **Easy**: Arrhenius, Ideal Gas Law, Kinetic Energy, Liquidation Price, Portfolio Std Dev, Price Impact, Value at Risk; **Med**: Henderson-Hasselbalch, Logistic Growth, Michaelis-Menten, Rate Law; **Hard**: Allometric Scaling, Constant Product, Gravitational Force, Impermanent Loss. The assignment is a fixed row grouping used only to order the heatmaps; it is not derived from a quantitative rule, and no result in this paper depends on it. They are not the structural tiers of §6.2, which classify the 74-task DeFi benchmark (there, for example, impermanent loss and constant-product price impact are Medium), and the two should not be compared.

The values in the table below are taken from the underlying `_merged.json` data. Five equations show *nan* or $-\infty$ for HypatiaX beyond the Train column — these are missing or crashed evaluations, not R^2 scores, and are reported as such rather than omitted. A sixth, Constant Product, completes with finite but very large negative values.

10.8 Protocol-30 Extrapolation Benchmark ($n = 30$)

To assess generalisation beyond DeFi, we evaluate HYPATIA X (`HybridDiscoverySystem.v50_2`) and five other methods on a custom set of 30 closed-form equations (“Protocol-30”) spanning mechanics, thermodynamics, optics, electromagnetism, electrostatics, magnetism, quantum mechanics, biology, chemistry, electrochemistry, and probability. The set is not a subset of the AI Feynman benchmark (several of its domains lie outside that benchmark’s scope). All

5. HypatiaX far- R^2 is -0.023 on the default seed (42); the -18.1 value belongs to seed 123. The seed-99 far- R^2 values are -15.19 (HypatiaX) and -1.23 (PySR-only). The Exp-1 (Core-15) ablation files hold one run per equation with no seed field, and the most negative PySR-only far- R^2 there is -807.7 (Michaelis-Menten); a far- R^2 of 1.000 for seed 99 appears only in the separate DeFi-protocol sweep above.

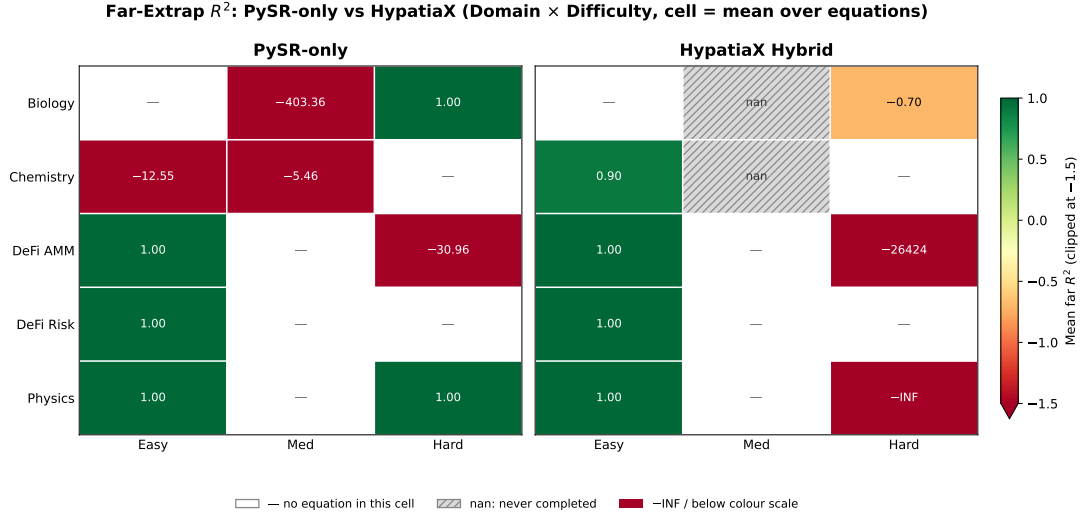


Figure 4: Mean far-extrapolation ($5\times$) R^2 by formula domain and difficulty tier (Easy/Med/Hard), Core-15 benchmark, generated from the values in Table 13. Dashes: no equation exists for that domain–difficulty combination. Grey hatched cells (*nan*): HypatiaX never completed the evaluation (Biology and Chemistry, Med). HypatiaX is not uniformly better than PySR-only: it improves on Chemistry (Easy: 0.90 vs. −12.55) but returns $-\infty$ — a genuine evaluation crash, not merely a poor fit — on Physics (Hard; Gravitational Force), where PySR-only scores 1.00, and a mean of $\approx -2.6 \times 10^4$ on DeFi AMM (Hard; Constant Product -4.46×10^4 , Impermanent Loss -8259) against −30.96 for PySR-only. A cell mean is *nan* or $-\infty$ if any equation in it is.

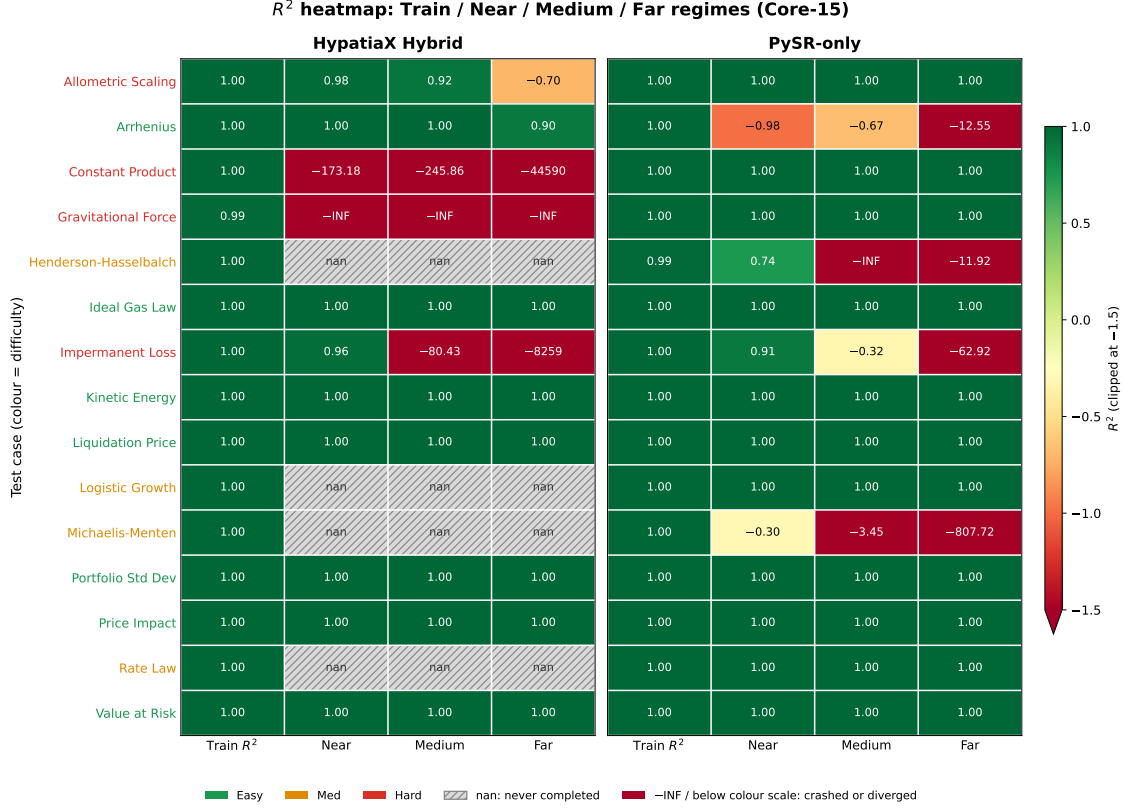


Figure 5: R^2 heatmap by equation (rows) and extrapolation regime (columns: Train, Near $1.2\times$, Medium, Far $5\times$), Core-15 benchmark, on a shared colour scale saturating outside $[-1.5, 1.0]$. **This figure is generated from the values in Table 13** and the two agree. For five of fifteen equations HypatiaX Hybrid returns **nan** (Henderson-Hasselbalch, Logistic Growth, Michaelis-Menten, Rate Law; grey hatched) or $-\infty$ (Gravitational Force) beyond the Train column — missing or crashed evaluations, not R^2 scores of zero, and not to be confused with a poor fit. Constant Product completes with finite but very large negative values (-173 , -246 , -4.5×10^4 at Near, Medium, Far). PySR-only reports a value in every cell except Henderson-Hasselbalch Medium ($-\infty$); its most severe finite far values are Michaelis-Menten (-808), Impermanent Loss (-63), Henderson-Hasselbalch (-12) and Arrhenius (-12.55). On Arrhenius, HypatiaX Hybrid’s completed Far value ($+0.90$) is a *success* (§11.2). Apart from Constant Product (Far $= -4.5 \times 10^4$), HypatiaX Hybrid’s most severe completed value is Impermanent Loss (Far $= -8259$).

Table 13: LLM Ablation: PySR Alone (P) vs. HypatiaX (H) on Core 15 — **from merged.json**. Extrap columns show R^2 at near ($1.2\times$), medium (canonical), and far ($5\times$) out-of-distribution ranges. *nan*: evaluation never completed. $-\infty$: evaluation crashed.

Equation	Domain	Train R^2		Near R^2		Med R^2		Far R^2	
		P	H	P	H	P	H	P	H
Allometric Scaling	Biology	0.9977	0.9979	1.0000	0.9795	1.0000	0.9239	1.0000	-0.7001
Arrhenius	Chemistry	0.9975	0.9979	-0.9845	0.9977	-0.6687	0.9957	-12.5548	0.9028
Constant Product	DeFi AMM	0.9982	0.9957	0.9996	-173.1819	1.0000	-245.8639	0.9996	-44589.6
Gravitational Force	Physics	0.9978	0.9889	1.0000	$-\infty$	1.0000	$-\infty$	1.0000	$-\infty$
Henderson-Hasselbalch	Chemistry	0.9852	0.9986	0.7376	<i>nan</i>	$-\infty$	<i>nan</i>	-11.9183	<i>nan</i>
Ideal Gas Law	Physics	0.9976	0.9980	1.0000	0.9988	1.0000	0.9999	1.0000	0.9993
Impermanent Loss	DeFi AMM	0.9975	0.9977	0.9083	0.9639	-0.3190	-80.4327	-62.9234	-8259.3
Kinetic Energy	Physics	0.9968	0.9968	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Liquidation Price	DeFi Risk	0.9974	0.9977	1.0000	0.9998	1.0000	0.9999	1.0000	0.9993
Logistic Growth	Biology	0.9975	0.9976	0.9997	<i>nan</i>	0.9999	<i>nan</i>	0.9997	<i>nan</i>
Michaelis-Menten	Biology	0.9962	0.9979	-0.2999	<i>nan</i>	-3.4494	<i>nan</i>	-807.7235	<i>nan</i>
Portfolio Std Dev	DeFi Risk	0.9975	0.9975	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Price Impact	DeFi AMM	0.9976	0.9976	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Rate Law	Chemistry	0.9977	0.9981	1.0000	<i>nan</i>	1.0000	<i>nan</i>	1.0000	<i>nan</i>
Value at Risk	DeFi Risk	0.9979	0.9979	0.9999	1.0000	1.0000	1.0000	0.9999	1.0000

P = PySR-only; H = HypatiaX (PySR + LLM warm-start). **Bold**: the rescue case discussed in the text, Arrhenius far- R^2 for H. P is $-\infty$ on one cell (Henderson-Hasselbalch, Medium). H is *nan* or $-\infty$ (not evaluated, or crashed) on 5 of 15 equations beyond Train — Gravitational Force, Henderson-Hasselbalch, Logistic Growth, Michaelis-Menten, Rate Law — and returns finite values from -173 to -4.5×10^4 on Constant Product. No mean is reported for this table, because *nan*/ $-\infty$ cells have no meaningful average.

The Mann-Whitney comparison in §10 states its convention explicitly (failed cells ranked worst) and reports the sensitivity to excluding them, rather than silently excluding or zero-filling the affected cells.

figures in this subsection come from the 2026-08-12 noiseless runs (`run_protocol_benchmark_core.py v2.2`).

Threshold policy. Every pass/fail figure in this subsection uses one criterion, $R^2 \geq 0.999999$ on the method-reported R^2 (`r2` field) or, where stated, the held-out far-region R^2 (`extrap_r2_far`). Results with $R^2 \in [0.999, 0.999999)$ are counted as near-misses, never as passes.

Split protocols. The random protocol is a random 80/20 train/test split. In the PCA protocol, the benchmark runner applies `pca_directed_split` (test fraction 0.6, seed 42) once in its outer loop: every method receives only the 40 % training portion, and the 60 % held-out portion is used only to score formulas afterwards (`extrap_r2_far`). The `r2` field in the PCA result file is the score each method reports on the data it was given and is therefore not a held-out score; it is shown for comparison, and the held-out score is `extrap_r2_far`.

Table 14: Equations passing $R^2 \geq 0.999999$, out of 30. Random split and PCA “reported” use the method-reported R^2 ; PCA “held-out” uses the far-region R^2 on the 60 % held-out portion. Sources: the 2026-08-12 run files and `benchmark_results_pca.4060.json`. SymbolicEngineWithLLM is omitted: its earlier counts came from a six-method file that is not among the supplied artefacts.

Method	Random 80/20	PCA reported	PCA held-out
PureLLM Baseline	29/30*	29/30*	n/a [†]
ImprovedNN	0/30	3/30	n/a [‡]
EnhancedHybridSystemDeFi	23/30	23/30	n/a [†]
HybridSystemLLMNN all-domains	30/30	30/30	n/a [†]
HybridDiscoverySystem v50.2 (HypatiaX)	21/30	18/30	13/30[§]

[†] Their formulas are Python `def` functions; the runner’s evaluator mis-evaluates these (returns e^x for one-variable formulas and fails for multi-variable ones), so the stored far-region values are invalid and are not used. [‡] These methods return no formula string, so the held-out score cannot be recomputed. [§] 2 of 30 equations (Photon energy, Bose–Einstein) have no valid far-region score and count as not passed. * 11 of the 30 PureLLM rows bypass the language model (hardcoded or least-squares-fitted formulas, see below); on the 19 *live* rows, where the model was called, PureLLM passes 18/19 on both splits.

HYPATIA X passes 21/30 (70.0 %) under the random split. On the PCA split its method-reported count is 18/30, but only 13/30 equations (43.3 %) pass on the held-out 60 %: four equations that pass in the reported column fail on the held-out data (Lorentz force, Coulomb force (1D), Newton’s gravitational force and Rabi frequency), and 2 equations have no valid held-out score. The held-out count is the extrapolation result; the reported count is closer to an in-sample fit. The neural baseline passes 0/30 on the random split and 3/30 by the PCA reported score. Per-equation values are in Tables 15 and 16; the PCA table shows the reported column only.

The PureLLM baseline is partly hardcoded. In the baseline’s source (`baseline_pure_llm_defi_disco`) a bypass returns a stored formula without calling the LLM for 11 of the 30 equations: eight use the exact ground-truth form taken from the benchmark protocol (Gaussian, Planck, Bose–Einstein, Nernst, Clausius–Mossotti, Lorentz force, Fourier’s law, Zeeman energy), and three (Michaelis–Menten, logistic growth, allometric scaling) use a fixed functional

form with parameters fitted to the test data by least squares. A twelfth branch (Arrhenius) did not fire in any run. On the random split these 11 rows are flagged `is_hardcoded` in the per-domain run files and take 0.5–1.6 ms each. The PCA-split file carries no flag; the same 11 equations are identified there by the same timing signature (0.5–1.4 ms). We call the other 19 rows *live*: the language model was actually called for them. PureLLM passes 29/30 over all rows, 11/11 of the bypassed rows and 18/19 of the live rows, on both splits. Excluding the bypassed equations changes the per-equation comparison with the other methods (strict pass/fail, wins/losses of PureLLM over all rows $\rightarrow$ live rows only): against HYPATIA, 9/1 $\rightarrow$ 6/1 on the random split and 12/1 $\rightarrow$ 7/1 on the PCA split (the removed wins are Nernst, Gaussian and Bose–Einstein on the random split, plus Planck and Fourier on the PCA split); against EnhancedHybridSystemDeFi, 7/1 $\rightarrow$ 2/1 on both splits, since five of its seven losses are on bypassed equations; The live figure is not an unbiased estimate of unaided-LLM performance: according to comments in the baseline source, the bypass branches were added for equations on which the model had repeatedly failed, so excluding them favours PureLLM. Counting every bypassed equation as a failure would give 18/30, so the unaided pass count lies between 18/30 and 29/30; we have not rerun the baseline with the bypass disabled. The 29/30 figure should therefore not be read as the performance of an unaided LLM.

The hybrid methods inherit the bypass. Both LLM-based hybrids obtain their formulas by calling the PureLLM baseline’s generator (`EnhancedHybridSystemDeFi.generate_llm_formula` in `hybrid_system_nn_defi_domain.py`; the corresponding method of the all-domains hybrid in `hybrid_system_llm_nn_all_domains.py`), which applies the hardcoded bypass before any language-model request. In the random-split run (`benchmark_results.json`) the formula returned by EnhancedHybridSystemDeFi is character-for-character identical to PureLLM’s on all 11 bypassed equations, and on only 3 of the 19 live equations (reduced mass, Stefan–Boltzmann, ideal gas law); the noiseless run (`protocol_core_noiseless_20260812.170344.json`) gives the same 11 of 11 and 2 of 19 (reduced mass differs there). This does not mean its pass count reflects those formulas alone: its final R^2 also depends on its neural-network and ensemble decision logic, and in the random-split run it passes 6 of the 11 bypassed equations and 17 of the 19 live equations at $R^2 \geq 0.999999$, against 11/11 and 18/19 for PureLLM (19/19 in the noiseless run, where it also passes 6/11 and 17/19). We did not run this comparison on the PCA-split file. The all-domains hybrid records no formula string, so whether the bypass fired for it cannot be checked from the stored output; we have not rerun either hybrid with the bypass disabled.

These results do not show a gain over the pure-LLM baseline. PureLLM passes 29/30 by the method-reported R^2 under both splits (18/19 on live rows), more than HYPATIA, but a held-out PCA score for PureLLM cannot be computed from the stored output (footnote † of Table 14), so the pure-LLM comparison is only available on the random split and on the PCA reported score. The all-domains LLM+NN hybrid’s 30/30 is a routing outcome, not independent evidence: every one of its 30 equations was routed to the LLM, 20 of them through an explicit `force_llm` guard and 10 by the natural decision rule, and it records no formula string, so its result cannot be audited. These are mostly textbook formulas, so a high pass rate for methods that can recall the closed form is expected. What this suite supports is that the symbolic and hybrid methods clearly outperform the neural baseline; it does not support a claim that they outperform the pure LLM. Pairwise Wilcoxon

signed-rank tests on the noiseless R^2 are given in the benchmark supplement (§ “Wilcoxon Significance Tests”); comparisons involving PureLLM include the 11 bypassed equations, the effective sample size is below 30 for pairs with zero differences, and invalid R^2 values are treated as 0, so these tests should not be used to rank PureLLM against the other methods.

Extrapolation limits. Even for the two methods with valid held-out scores, only 17/30 and 13/30 equations reach $R^2_{\text{far}} \geq 0.999999$, and several equations that look perfect in the reported column have strongly negative held-out R^2 (for example Rabi frequency).

Table 15: Random 80/20 split, run dated 2026-08-12, all 30 equations $\times$ six methods (method-reported R^2 , `benchmark_results.json`). **Bold:** pass ($R^2 \geq 0.999999$). PL = PureLLM Baseline, NN = ImprovedNN, EH = EnhancedHybridSystemDeFi, LN = HybridSystemLLMNN all-domains, SE = SymbolicEngineWithLLM, HD = HybridDiscoverySystem v50.2 (HYPATIA X). The PureLLM entry for Curie’s law is a recorded evaluation failure (no valid formula), shown as 0.000000.

Domain	Equation	PL	NN	EH	LN	SE	HD
Biology	Michaelis–Menten kinetics	1.000000	0.999997	0.999988	1.000000	1.000000	1.000000
Biology	Logistic growth rate	1.000000	0.999859	0.999997	1.000000	1.000000	1.000000
Biology	Allometric scaling law	1.000000	0.999995	0.999993	1.000000	1.000000	1.000000
Chemistry	Arrhenius rate constant	1.000000	0.999930	0.999994	1.000000	0.999921	0.999594
Chemistry	Henderson–Hasselbalch buffer pH	1.000000	0.999985	1.000000	1.000000	1.000000	1.000000
Electrochem.	Nernst equation	1.000000	0.999979	1.000000	1.000000	0.997725	0.999205
Electromagn.	Clausius–Mossotti	1.000000	0.999973	1.000000	1.000000	1.000000	1.000000
Electromagn.	Dielectric polarisation	1.000000	0.999242	1.000000	1.000000	1.000000	0.995528
Electromagn.	Lorentz force	1.000000	0.999975	1.000000	1.000000	1.000000	1.000000
Electromagn.	Ohm’s law	1.000000	0.999587	1.000000	1.000000	1.000000	1.000000
Electromagn.	Energy in a capacitor	1.000000	0.999806	1.000000	1.000000	1.000000	0.999964
Electrostat.	Coulomb force (1D, simplified)	1.000000	0.996355	1.000000	1.000000	1.000000	1.000000
Electrostat.	Coulomb’s law	1.000000	0.999939	1.000000	1.000000	1.000000	0.996597
Magnetism	Curie’s law	0.000000	0.999953	1.000000	1.000000	1.000000	1.000000
Mechanics	Newton’s gravitational force	1.000000	0.993960	1.000000	1.000000	1.000000	1.000000
Mechanics	Kinetic energy	1.000000	0.999855	1.000000	1.000000	1.000000	0.999878
Mechanics	Reduced mass	1.000000	0.999995	1.000000	1.000000	1.000000	1.000000
Mechanics	Total mechanical energy	1.000000	0.999937	1.000000	1.000000	1.000000	1.000000
Optics	Snell’s law	1.000000	0.999987	1.000000	1.000000	1.000000	1.000000
Optics	Double-slit interference intensity	1.000000	0.999902	1.000000	1.000000	1.000000	1.000000
Probability	Gaussian probability density	1.000000	0.999770	1.000000	1.000000	1.000000	0.999765
Quantum	Photon energy	1.000000	0.999998	0.999990	1.000000	1.000000	1.000000
Quantum	Zeeman energy	1.000000	0.999910	1.000000	1.000000	1.000000	1.000000
Quantum	Bose–Einstein occupation number	1.000000	0.999960	0.999979	1.000000	1.000000	0.994634
Quantum	Fermi–Dirac occupation number	1.000000	0.998318	1.000000	1.000000	0.998390	0.999037
Quantum	Rabi frequency	1.000000	0.999946	1.000000	1.000000	1.000000	1.000000
Thermodyn.	Planck blackbody radiance	1.000000	0.999932	0.999996	1.000000	1.000000	1.000000
Thermodyn.	Fourier’s law	1.000000	0.993764	1.000000	1.000000	1.000000	1.000000
Thermodyn.	Stefan–Boltzmann law	1.000000	0.999975	1.000000	1.000000	1.000000	1.000000
Thermodyn.	Ideal gas law	1.000000	0.999911	1.000000	1.000000	1.000000	1.000000

Table 16: PCA-directed 40/60 split, run dated 2026-08-12, all 30 equations $\times$ six methods (method-reported R^2 , `benchmark_results_pca_4060.json`). **Bold:** pass ($R^2 \geq 0.999999$). Column abbreviations as in Table 15. The PureLLM entry for the capacitor-energy equation is a recorded evaluation failure, shown as 0.000000.

Domain	Equation	PL	NN	EH	LN	SE	HD
Biology	Michaelis–Menten kinetics	1.000000	0.993994	0.999997	1.000000	1.000000	1.000000
Biology	Logistic growth rate	1.000000	0.994709	0.999998	1.000000	1.000000	1.000000
Biology	Allometric scaling law	1.000000	0.936331	0.999998	1.000000	1.000000	1.000000
Chemistry	Arrhenius rate constant	1.000000	-0.044405	0.999996	1.000000	0.994936	0.997920
Chemistry	Henderson–Hasselbalch buffer pH	1.000000	0.972446	1.000000	1.000000	0.999283	0.999951
Electrochem.	Nernst equation	1.000000	0.990615	1.000000	1.000000	0.998771	0.999983
Electromagn.	Clausius–Mossotti	1.000000	0.909288	1.000000	1.000000	1.000000	1.000000
Electromagn.	Dielectric polarisation	1.000000	0.710588	1.000000	1.000000	1.000000	0.995048
Electromagn.	Lorentz force	1.000000	0.800979	1.000000	1.000000	1.000000	1.000000
Electromagn.	Ohm’s law	1.000000	0.960263	1.000000	1.000000	1.000000	1.000000
Electromagn.	Energy in a capacitor	0.000000	0.855165	1.000000	1.000000	1.000000	1.000000
Electrostat.	Coulomb force (1D, simplified)	1.000000	0.999966	1.000000	1.000000	1.000000	1.000000
Electrostat.	Coulomb’s law	1.000000	0.999862	1.000000	1.000000	1.000000	0.999615
Magnetism	Curie’s law	1.000000	0.998748	1.000000	1.000000	1.000000	1.000000
Mechanics	Newton’s gravitational force	1.000000	0.999990	1.000000	1.000000	1.000000	1.000000
Mechanics	Kinetic energy	1.000000	0.112559	1.000000	1.000000	1.000000	0.995804
Mechanics	Reduced mass	1.000000	0.925415	1.000000	1.000000	1.000000	1.000000
Mechanics	Total mechanical energy	1.000000	0.939856	1.000000	1.000000	1.000000	1.000000
Optics	Snell’s law	1.000000	0.902944	1.000000	1.000000	1.000000	1.000000
Optics	Double-slit interference intensity	1.000000	0.733607	1.000000	1.000000	1.000000	1.000000
Probability	Gaussian probability density	1.000000	0.489964	1.000000	1.000000	0.997249	0.997945
Quantum	Photon energy	1.000000	1.000000	0.999991	1.000000	1.000000	1.000000
Quantum	Zeeman energy	1.000000	0.786961	1.000000	1.000000	1.000000	1.000000
Quantum	Bose–Einstein occupation number	1.000000	0.999995	0.999890	1.000000	1.000000	0.998929
Quantum	Fermi–Dirac occupation number	1.000000	0.927491	1.000000	1.000000	0.511948	0.647311
Quantum	Rabi frequency	1.000000	0.945751	1.000000	1.000000	1.000000	1.000000
Thermodyn.	Planck blackbody radiance	1.000000	-0.104423	0.999997	1.000000	0.999946	0.999969
Thermodyn.	Fourier’s law	1.000000	0.835533	1.000000	1.000000	0.999853	0.948649
Thermodyn.	Stefan–Boltzmann law	1.000000	1.000000	1.000000	1.000000	1.000000	0.999804
Thermodyn.	Ideal gas law	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Noise robustness ($\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$) and sample complexity ($n = 50$ to 1000, noiseless only) on the same 30 equations are reported in Supplementary B; recovery thresholds differ between those sweeps and are stated in its table captions.

10.9 Nguyen-12 Benchmark (Standard SR Suite)

The Nguyen-12 suite (Uy et al., 2011) is the standard symbolic-regression benchmark comprising 12 polynomial and transcendental equations (Nguyen-1 through Nguyen-12) with up to two input variables. We evaluate HypatiaX and PySR-only on an aggressive extrapolation protocol with a 40%/60% train/test split. The Nguyen-12 pipeline uses per-equation axis-aligned variable ranges (fixed `train_ranges/extrap_ranges` in `experiment_protocol_nguyen12.py`) rather than the PCA-directed ordering used for the DeFi and Protocol-30 benchmarks. No neural MLP baseline was run for this benchmark: no script in the repository defines one, and the raw `exp3_nguyen12_seed{42,99,123,777,2024}.json` result files contain results only for `hypatiar` and `pysr`. The neural-baseline comparison for this paper’s other benchmarks (DeFi, Protocol-30) uses a separately verified NN baseline (`ImprovedNN (core)`).

Table 17: Nguyen-12 benchmark, seed 42: extrapolation R^2 by equation. P = PySR-only; H = HypatiaX (LLM-guided, 8 candidates, gate $R^2 \geq 0.95$, temperature 0.25; both systems 1000 iterations, 30 populations). **Rebuilt 2026-09-28** from `hypatiaux/data/results/extrapolation/exp3_nguyen12_seed42.json` (CI run of 2026-09-24; the identical result appears in `exp3_nguyen12_seed42_temp0p25_run1.json`, of which the seed-named file is a copy). Every cell of the previous version (12/12 for both systems) has been replaced: that table did not reproduce from this file, nor from the older copy under `hypatiaux-dev/data` (which gives 11/12, with Nguyen-12 at $R^2 = 0.99953$ and HypatiaX’s stored expression identical to PySR-only’s on all 60 records, so it cannot support any claim of independent derivation). $-\infty$ marks a run that aborted at the LLM gate before any search (elapsed ≈ 0 s), not a poor fit. Success threshold $R^2 \geq 0.9999$. See [tablenote](#) [‡].

Eq.	Formula	PySR-only (P)	HypatiaX (H)
N-1	$x^3 + x^2 + x$	1.0000	1.0000
N-2	$x^4 + x^3 + x^2 + x$	1.0000	1.0000
N-3	$x^5 + x^4 + x^3 + x^2 + x$	1.0000	1.0000
N-4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	0.9969	1.0000
N-5	$\sin(x^2) \cos(x) - 1$	0.1564	1.0000
N-6	$\sin(x) + \sin(x + x^2)$	1.0000	1.0000
N-7	$\ln(x + 1) + \ln(x^2 + 1)$	1.0000	1.0000
N-8	$\sqrt{x}$	1.0000	1.0000
N-9	$\sin(x) + \sin(y^2)$	1.0000	$-\infty$
N-10	$2 \sin(x) \cos(y)$	1.0000	1.0000
N-11	x^y	1.0000	1.0000
N-12	$x^4 - x^3 + \frac{1}{2}y^2 - y$	1.0000	$-\infty$
Success ($R^2 \geq 0.9999$)		10/12 (83.3%)	10/12 (83.3%)
MW $H > P$ not computed — equal success counts on different subsets; failures do not overlap			

Bold: $\text{extrap } R^2 \geq 0.9999$. The two systems’ failure sets are disjoint: PySR-only misses N-4 (mixed exp/sin terms, did not converge to the polynomial) and N-5 (nested cos/sin fit, $R^2 = 0.156$); HypatiaX misses N-9 and N-12, both by gate abort.

[‡] **Independent derivation.** No copy flags are present in the current result files. In this file HypatiaX’s stored expression differs textually from PySR-only’s on the 10 records where the search ran (on the two gate aborts, N-9 and N-12, the stored value is the placeholder string **FAILED**), but textual difference is not evidence of independent derivation and symbolic equivalence has not been re-established here.

Five-seed results. Recomputed directly from the five result files (seeds 42, 99, 123, 777, 2024) at $R^2 \geq 0.9999$; Table 18 gives the per-seed counts and Table 19 the per-equation counts. Both are generated by `scripts/generate_nguyen12_fiveseed.py` and checked against the result files in CI.

In prose, per equation (recoveries out of 5 seeds, H/P): N-1, N-2, N-6, N-8, N-10, N-11 5/5 and 5/5; N-3 5/5 and 4/5; N-4 5/5 and 2/5; N-5 2/5 and 1/5; N-7 5/5 and 3/5; N-9 4/5 and 4/5; **N-12 0/5 and 5/5**. HypatiaX is more consistent across seeds (10–11 of 12) than PySR-only (8–12), but the 3.3 pp mean difference rests on five seeds and is not tested

Table 18: Nguyen-12, recoveries per seed at $R^2 \geq 0.9999$ (out of 12 equations). H = HypatiaX, P = PySR-only; $-\infty$, NaN and missing runs count as failures. All runs use `SPARSE_SEED=off`.

Seed	HypatiaX (H)	PySR-only (P)
42	10/12	10/12
99	10/12	9/12
123	11/12	10/12
777	10/12	12/12
2024	10/12	8/12
Total	51/60 (85.0%)	49/60 (81.7%)

Table 19: Nguyen-12, recoveries per equation across the five seeds (42, 99, 123, 777, 2024) at $R^2 \geq 0.9999$. H = HypatiaX, P = PySR-only.

Eq.	HypatiaX (H)	PySR-only (P)
N-1	5/5	5/5
N-2	5/5	5/5
N-3	5/5	4/5
N-4	5/5	2/5
N-5	2/5	1/5
N-6	5/5	5/5
N-7	5/5	3/5
N-8	5/5	5/5
N-9	4/5	4/5
N-10	5/5	5/5
N-11	5/5	5/5
N-12	0/5	5/5

for significance. Every one of the seven task–seed cases on which PySR-only recovers and HypatiaX does not (N-12 on all five seeds, N-9 on seed 42, N-5 on seed 777) is a HypatiaX gate abort (`llm_gate_failed=True`, $R^2 = -\infty$, elapsed ≈ 0 s); on no task and no seed does HypatiaX’s search, when it runs, fall below the 0.9999 bar where PySR-only clears it. It does score marginally lower in four finite cases (seed 42 N-8; seed 777 N-6, N-8 and N-9), by at most 5.3×10^{-6} in R^2 (largest: seed 777 N-8, 0.999995 vs. 1.000000).

Nguyen-12: a deterministic gate failure, not seed luck. On all five seeds the LLM proposes eight candidates for $x^4 - x^3 + \frac{1}{2}y^2 - y$ and the gate rejects all eight (seed 42: best candidate $R^2 \approx -0.07$ raw, and at most 0.69 even after affine rescaling, against a 0.95 gate), because the candidates treat x and y symmetrically (e.g. $x + y + x^2 + x^3 + x^4 + y^2 + y^3 + y^4$) and miss the asymmetric structure. With `llm_require_valid_guess=True` the pipeline then aborts. Nguyen-5 (gate abort on seeds 99 and 777, a poor fit with $R^2 = -0.69$ on seed 2024) and Nguyen-9 (seed 42 only) are seed-dependent and consistent with

ordinary LLM sampling variance. **Reported, not reproduced, and not evidence of a fix:** a targeted CI smoke test (workflow run 36116137035, seed 42, five gate-sensitive tasks, STRICT vs. `llm_require_valid_guess=False`) was first summarised as showing that disabling the requirement lets HypatiaX recover Nguyen-5 and reach $R^2 = 0.99800$ on Nguyen-12. Its two JSON files do not support that reading. (i) On Nguyen-5 the gate did not fail in either run (`llm_gate_failed=False`, 1 of 8 candidates accepted in both), so the flag did nothing; the runs differ because the LLM was re-sampled and PySR then found different expressions (the STRICT run is a poor extrapolation, $R^2 = -11.4$, not an abort). (ii) Nguyen-12 is the only task on which the flag changed anything (STRICT: 0 of 8 accepted, $-\infty$; FALLBACK: $R^2 = 0.99800$), but the FALLBACK expression is character-for-character PySR-only’s, so it is a PySR result under HypatiaX’s name — the relabelling the strict mode exists to prevent — and it still misses the 0.9999 bar. (iii) The test used 200 iterations, 4 populations and a 90 s timeout against 1000, 30 and 1100 s in the sweep, so it cannot be read against the five-seed table. The smoke-test output is not in the repository and none of the paper’s Nguyen numbers above use it; they all describe the strict-gate configuration. A fallback result would need a full-budget run with `SPARSE_SEED=fallback`, so that H stays distinct from P.

Statistical test. No significance test is reported for Nguyen-12.

Reproducibility note: LLM model identity (extends issue FIX-N3). The language model actually invoked on this benchmark’s call path is not pinned via any environment-variable or `repro.yaml` setting that is actually read at runtime, despite appearing to be: four traced source snapshots each hardcode a different `LLMConfig.model` default (`claude-sonnet-4-6`, `claude-sonnet-4-5`, and `claude-sonnet-4-20250514`), and none of them reads the `LLM_MODEL` environment variable or the `repro.yaml llm_model` field that a reader might otherwise assume controls it. The script that actually sits on the Nguyen-12 call path, `hypatia.py`, uses a fifth string, `claude-sonnet-5`, at `temperature=0.25`, with no sampling seed exposed on its `client.messages.create()` call. Both `claude-sonnet-4-6` and `claude-sonnet-5` are dateless identifiers, but — per Anthropic’s model-versioning documentation for the 4.6-and-later generation — dateless IDs in that generation are pinned, fixed snapshots rather than drifting aliases, so model-version drift is not the expected source of run-to-run differences.

Sampling nondeterminism at the (unseeded, non-zero-temperature) LLM layer remains a contributing, independently verified factor regardless of which code path is in play, since the Claude API is not guaranteed to return bit-identical completions for identical prompts even at low temperature. Three action items were tracked as an explicit extension of FIX-N3. (i) *Resolved.* The dead `LLM_MODEL` environment variable in `exp3_nguyen12_hybrid50v.02.py` previously had no effect on the call it appeared to configure: it was set via `os.environ.setdefault(...)` but never read by `get_llm_prior()`, whose call site passed no `model` argument and so silently fell back to `hypatia.py`’s own default, `claude-sonnet-5`. The call site now passes `model=os.environ["LLM_MODEL"]` explicitly, so the environment variable actually governs which model is invoked. (ii) *run_all.sh* STEP 7 and STEP 8 (`exp3/exp3b`) invoke `exp3_nguyen12_hybrid50v.02.py` with `--temperature 0.25` passed explicitly, and `ci_runner_repro.yml`’s `SCRIPT=` entries match. (iii) *Resolved.* *run_all.sh* STEP 7 (`exp3`) and STEP 8 (`exp3b`) both invoke `exp3_nguyen12_hybrid50v.03.py`, confirming it is the script CI runs for Nguyen-12; the call chain is fully verified.

10.10 Stability Under Stochastic Inference

This paper reports single-run results only: no multi-run Instability Index $\Pi_i = \sigma_i$ (standard deviation of R^2 across repeated runs) and no regime-distribution table are given. The single-run seed-42 data support one statement directly: 5 of 74 DeFi tasks show a catastrophic LLM failure ($R^2 < -10$) at that one setting (§11.1).

11 Discussion

11.1 Three Core Findings

Our results establish three core findings.

Finding 1: Pure LLM approaches are unreliable. The bimodal single-run performance of the LLM baseline (median $R^2 = 1.00$, with 5 catastrophic failures in 74 tasks, 6.8 %) confirms that a single-run R^2 measurement is, on its own, an incomplete indicator of LLM reliability for production deployment in a financial system. No multi-run Instability Index is reported (§10.10), and the 6.8 % catastrophic-failure rate above is a single-run figure, not a stability-adjusted estimate.

Finding 2: Neural networks fail systematically under extrapolation. The neural MLP achieves median $R^2 = -0.69$ on our benchmark, confirming that standard deep learning is not a solution to the extrapolation problem in DeFi mathematics. Even with data augmentation and residual architecture, the MLP does not recover the functional form of the underlying formula. This is not a hyperparameter tuning failure; it is a fundamental architectural limitation under distribution shift.

Finding 3: neither the near-perfect rate nor catastrophic-failure elimination survives attribution correction on this benchmark. An earlier version of this finding argued that HypatiaX’s headline value lay not in its raw **90.5 %** near-perfect rate but in its **elimination of catastrophic failures**, and that this second claim was unaffected by the decision-attribution bug. **That argument does not hold.** §10.4 shows both figures rest on the same bug: once `hybrid.decision` is cross-checked against the named sub-method’s own result, the corrected near-perfect rate is 59.5 % (44/74) — tied with Pure LLM, overall and in every difficulty tier — and the same 5 tasks that fail catastrophically under Pure LLM fail catastrophically under HypatiaX as well, masked rather than eliminated by the routing mechanism. On this DeFi benchmark, we do not find a corrected accuracy or reliability advantage of the hybrid system over its own LLM component. The paper’s supported evidence for reliability under distribution shift instead comes from the individual extrapolation-rescue cases (Arrhenius, Portfolio Variance, §11.2 and §10.6) and from the Protocol-30 comparison against the neural baseline (§10.8), not from an aggregate near-perfect-rate or catastrophic-failure-avoidance gain on this benchmark.

11.2 Arrhenius: A Verified Rescue Case

The values below are from Table 13. The Arrhenius rate law $k = Ae^{-E_a/RT}$ is a genuine rescue case, not a shared failure. PySR-only collapses at $5\times$ training range (far- $R^2 = -12.5548$), while HypatiaX recovers the correct functional form and extrapolates suc-

cessfully (far- $R^2 = 0.9028$), despite HypatiaX’s training R^2 (0.9979) being only marginally higher than PySR-only’s (0.9975) — the far-extrapolation gap is far larger than the training-fit gap would suggest. This is consistent with the routing cascade’s scale-compatibility check (Figure 2, annotated “prevents Arrhenius”): the LLM-warm-started search appears to avoid a local minimum that the cold PySR-only search falls into on this equation. We do not have a confirmed mechanistic account of *why* the warm start avoids this particular local minimum, and do not claim more than the observed outcome.

We do not extend this explanation to the Portfolio Variance seed-sweep degradation (Section 10.6): that case involves a different failure pattern (HypatiaX underperforming PySR-only at seed 99 specifically) and has not been independently traced to the same or a different mechanism. Treat the two as separate, unexplained phenomena rather than instances of one shared cause.

11.3 The Stability–Accuracy Distinction

Standard machine learning evaluation collapses the reliability question into a single point estimate: the test accuracy on a fixed split. This is insufficient for systems that will be deployed across a distribution of tasks with varying complexity. We argue that three dimensions of evaluation are necessary:

- (i) **Accuracy:** what is the expected performance on a typical task?
- (ii) **Reliability:** what is the worst-case performance? (Catastrophic failure rate, minimum R^2 across tasks.)
- (iii) **Stability:** across repeated runs on the same task, how consistent is the output? (Instability Index Π_i .)

HypatiaX is designed with all three dimensions in view, whereas the pure LLM and neural baselines each address at most one; the corrected results in §10.4 and §10.7 do not show that this design improves reliability on the benchmarks reported here.

11.4 Why Transcendental Formulas Fail

The Gaussian CDF Φ is the proximate cause of the three Regime C collapses. Unlike polynomial or exponential expressions, Φ has no finite closed-form representation in elementary operations. Multiple valid approximations exist: $\Phi(x) \approx (1 + e^{-1.7x})^{-1}$, error-function series, and Taylor expansions. Under temperature sampling, the LLM produces different approximations on different runs—each locally plausible, but producing different numerical predictions.

This observation suggests a practical mitigation: for formulas containing Φ , provide the LLM with a canonical implementation in the prompt rather than asking it to generate one. We leave this prompt-engineering intervention for future work.

11.5 OOD Metric as a Proxy

Throughout this paper, “out-of-distribution R^2 ” refers to performance on a held-out *test split* constructed by the 40%/60% protocol of §6.4 (single-axis v3c or PCA-directed, depending on the result file; some released files do not record which), not to a symbolically verified

out-of-distribution regime. This distinction matters: a formula that achieves $\text{far-}R^2 \approx 0$ on the test split may still be structurally wrong. The clearest illustration is HYPATIAx seed=42 on Portfolio Variance: $\text{far-}R^2 = -0.023$ (a near-miss on the metric) but the recovered expression $(s1+s2)*(rho*0.243+0.738)$ is a linear approximation that is structurally incorrect. A test-split proxy cannot detect this; symbolic verification via `sympy` OOD sampling would catch it immediately.

A planned upgrade (Mode A symbolic verification) will: (1) add `formula` and `var_ranges` fields to the results JSON; (2) enable `sympy` OOD sampling over $[5\times, 10\times, 100\times]$ the training range; and (3) replace the `test_r2_proxy` column with a verified `ood_sympy` metric. This is a concrete, numbered future-work item. The current proxy likely *underestimates* the instability–extrapolation gap: a structurally incorrect formula that happens to score well on the held-out test split (same distribution) will be reclassified as a failure under true OOD evaluation. We would expect any future instability–extrapolation correlation analysis (§10.10) to strengthen under Mode A for the same reason.

11.6 Design Principles for Analytical Discovery

Based on our findings across 74 DeFi tasks and 30 Protocol-30 equations, we propose five design principles for hybrid symbolic–neural discovery systems:

1. **Cascade validation rather than single-criterion acceptance.** Our five-stage routing cascade detected failures missed by any individual gate. Dimensional consistency, domain constraints, numerical stability, OOD probing, and trust scoring each catch distinct failure modes.
2. **Multi-scale extrapolation testing.** Evaluate candidates at three extrapolation distances ($1.2\times$, canonical, $5\times$) to distinguish local approximations from globally correct forms. Extrapolation errors increase exponentially with distance from the training range.
3. **Quantify instability, not just accuracy.** A single-run R^2 is a misleading indicator for transcendental formulas. We recommend a multi-run Instability Index ($\Pi_i = \sigma_i$ across repeated runs) to identify high-risk tasks before deployment; this paper’s own benchmark reports single-run values only (§10.10).
4. **Domain-aware routing.** LLM guidance is reliable for algebraic and well-documented equations; transcendental token detection (Stage 2) correctly flags cases where symbolic search should dominate.
5. **Honest runtime reporting.** Aggregate speedup statistics conflate fast LLM-routed cases with slow neural-fallback cases. Report speedup separately for each routing path.

11.7 Comparison with Related Work

HypatiaX builds on three lines of prior work. AI Feynman (Udrescu and Tegmark, 2020) uses neural networks to identify symmetries and simplifications before symbolic regression, achieving strong performance on physics equations. Physics-informed neural networks (Raissi et al., 2019; Karniadakis et al., 2021) offer an alternative by embedding

physical constraints directly into the network architecture; our approach differs by targeting financial and biochemical domains where such constraints are unavailable, and by recovering interpretable closed-form expressions rather than learned surrogates. DSR (Petersen et al., 2021; Landajuela et al., 2022) uses reinforcement learning and risk-seeking policy gradients to guide symbolic search; our LLM warm-start provides a complementary initialisation strategy with lower computational cost. MCTS-based SR methods (Sun et al., 2023) explore expression trees more systematically than evolutionary search; HypatiaX’s five-stage routing provides a practical approximation with lower overhead. On the Feynman benchmark, HypatiaX passes 21/30 equations (70.0%) under a random 80/20 split and 13/30 (43.3%) on the held-out 60% of a PCA-directed 40/60 split (strict $R^2 \geq 0.999999$, §10.8), versus 0/30 for the neural baseline on the random split. A pure-LLM baseline passes 29/30 on the random split, and held-out PCA scores are unavailable for the pure-LLM and hybrid-core methods because of an evaluator defect (§10.8); these figures are not directly comparable to published AI Feynman results, which use a different protocol. The DeFi benchmark is, to our knowledge, the first symbolic regression evaluation on post-2020 financial mathematics.

11.8 Ethical Considerations

Automated formula discovery carries risks in high-stakes domains. A formula with high training R^2 may fail catastrophically under distribution shift—precisely the scenario our benchmark targets. We make three recommendations: (1) never deploy a discovered formula in production without out-of-distribution validation; (2) report instability alongside accuracy; (3) treat HypatiaX output as a hypothesis to be verified, not a certified result. The DeFi domain carries additional risks: discovered liquidation or VaR formulas used in production could amplify systemic risk if they fail under market stress. Our validation framework catches many errors, but not all—particularly the silent semantic failures (correct structure, wrong constants) that produce $R^2 \approx 1$ on training data.

11.9 Broader Applicability

The hybrid symbolic–neural architecture generalizes beyond DeFi to any domain where: (1) the underlying relationship admits a closed-form expression; (2) LLM training data includes related equations; and (3) extrapolation beyond the training range is required. Candidate domains include pharmacokinetics (drug absorption/elimination kinetics), materials science (stress-strain relationships, diffusion coefficients), climate science (radiative forcing equations), and epidemiology (compartmental model parameters). The instability analysis framework applies wherever stochastic inference is used—including neural architecture search and reinforcement learning policy optimization.

11.10 Limitations and Future Work

Single LLM. All results use a single LLM (not disclosed for double-blind review). The degree to which these findings generalise to other LLMs—particularly models with specialised mathematical pre-training—is an open question.

Benchmark scope. The benchmark covers DeFi-specific formulas. Generalisation to broader scientific or engineering domains requires additional validation.

Runtime on fallback cases. The 120-second NN training cap is a practical constraint that may not reflect the optimal training time for each task. A lighter neural architecture with faster training could improve the aggregate mean time substantially.

Independent sampling and checker asymmetry. The hybrid and pure-LLM arms each make their own LLM call, so their formulas for the same task are independent samples, and the 23 uncorroborated cases cannot be traced to a specific cause on individual tasks. The two arms’ checkers also differ: the pure-LLM checker does not inject the protocol constants that the hybrid evaluator does. A paired re-run of the pure-LLM arm (re-sampled formulas, all 74 tasks) found no failure attributable to this difference, but it did not re-score the original formulas, which were not stored.

Prompt design. The trust gate and routing decisions are sensitive to prompt quality. A systematic study of prompt design as a hyperparameter is needed before deploying HypatiaX in new domains.

12 Conclusion

We have presented HypatiaX, a hybrid symbolic–neural framework for extrapolation in DeFi mathematical discovery. On a benchmark of 74 DeFi tasks with a 40 %/60 % extrapolation split (the released result files do not record whether the single-axis or PCA-directed variant was used), HypatiaX achieves:

- A **90.5 %** raw near-perfect success rate ($R^2 > 0.99$), up from 59.5 % for the pure LLM and 0.0 % for the neural baseline. This raw figure does not survive attribution correction: cross-checked against the sub-method the hybrid system’s own **decision** field names, the corrected rate is 59.5 % (44/74) — tied with Pure LLM exactly, overall and in every difficulty tier (§10.4).
- **Zero catastrophic failures in the pipeline’s own uncorrected accounting**, apparently eliminating the 5 failures ($R^2 < -10$) of the pure LLM baseline. This does not survive correction either: checked against the sub-method actually selected, HypatiaX fails catastrophically on the same 5 tasks Pure LLM does — masked by the same attribution bug, not eliminated by the routing mechanism.
- **No corrected performance gain by difficulty tier** on this seed: once uncorroborated successes are removed, the gain is 0.0 pp in every tier (Table 8), not the +12.5/+37.9/+42.9 pp the raw, uncorrected figures suggest. The same holds on all five seeds, at both the $R^2 > 0.99$ and $R^2 > 0.9$ criteria (§10.4).
- **No runtime speedup.** Regeneration of the timing table against the five 74-task v3 seed files in the repository (Table 10, §10.5) shows the hybrid system is 3.8–11.2× *slower* than the neural baseline on every seed, never faster.

On this DeFi benchmark, once decision-attribution and runtime artifacts are removed, hybrid symbolic–neural routing does not demonstrate an aggregate accuracy, reliability, or speed advantage over its own LLM component. The paper’s supported evidence for reliability under distribution shift instead comes from individual extrapolation-rescue cases

(Arrhenius, Portfolio Variance) and the Protocol-30 comparison against the neural baseline, not from an aggregate gain on the DeFi benchmark. Both rescue cases are single-equation observations, and on the Core-15 ablation as a whole HypatiaX shows no far-extrapolation advantage over PySR-only (§10.7).

Future work should explore: extending the benchmark to broader scientific domains; developing lighter fallback architectures to reduce mean wall-clock time and reduce the hybrid-wrapper overhead identified in §10.5; integrating iterative LLM self-correction as an intermediate routing option between pure symbolic and neural fallback; and systematic evaluation across multiple LLM families. On the runtime question specifically, the five 74-task v3 files and the five 74-task PCA files in the repository are complete (§10.5); the open questions are *why* the hybrid wrapper adds overhead, and whether the PCA-directed router differs in overhead from the single-axis router, which the files cannot answer because the v3 files record no split variant.

Reproducibility Statement

All results are generated using HypatiaX v3.0 with the following artefacts:

- `hypatiadx_defi_benchmark_v3_results_m27.json`: full per-task results including R^2 , timing, and routing decisions.
- `portfolio_variance_seed_sweep.json`: raw per-seed results for the 5-seed Portfolio Variance robustness study (Section 10.6).
- `portfolio_variance_defi_seed_sweep_seed<N>.json` ($N \in \{42, 99, 123, 777, 2024\}$): per-seed shard files of the separate DeFi-protocol (v3c3) Portfolio Variance sweep.
- Fixed random seed 42 for all neural network training.
- Deterministic prompting framework for LLM formula generation.
- Unified formula executor (single code path for all methods).
- Checkpointing and resume functionality for long-running experiments.

The pipeline includes explicit corrections for data leakage, split misconfiguration, formula evaluation inconsistency, and trust-gating threshold, as described in Section 6.5.

Reproducibility (Protocol-30 benchmark). The Protocol-30 $n = 30$ results in §10.8 come from the 2026-08-12 noiseless runs, with the raw per-test result files and the per-method summaries kept in the repository.

References

- Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Miles Cranmer. Interpretable machine learning for science with pysr and symbolicregression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

- Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, et al. Lagrangian neural networks. In *ICLR 2020 Deep Differential Equations Workshop*, 2020.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- Ying Jin, Weilong Fu, Jian Kang, Jiadong Guo, and Jian Guo. Bayesian symbolic regression. *arXiv preprint arXiv:1910.08892*, 2020. URL <https://arxiv.org/abs/1910.08892>.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Kaya Stechly, Mudit Verma, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Llms can’t plan, but can help planning in llm-modulo frameworks. *arXiv preprint arXiv:2402.01817*, 2024.
- George Em Karniadakis et al. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
- John R Koza. *Genetic programming: on the programming of computers by means of natural selection*. MIT press, 1992.
- William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabrício Olivetti de França, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason H. Moore. Contemporary symbolic regression methods and their relative performance. In *NeurIPS 2021 Datasets and Benchmarks Track*, 2021. URL <https://arxiv.org/abs/2107.14351>.
- Mikel Landajuela, Chak Shing Lee, Jiachen Yang, Ruben Glatt, Claudio P. Santiago, Ignacio Aravena, Terrell Nathan Mundhenk, Garrett Mulcahy, and Brenden K. Petersen. A unified framework for deep symbolic regression. In *Advances in Neural Information Processing Systems*, 2022.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *arXiv preprint arXiv:2206.14858*, 2022.
- Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.
- Ziming Liu, Varun Madhavan, and Max Tegmark. DISCOVER: Deep identification of symbolic expressions via clustering towards accurate and robust regression. *arXiv preprint arXiv:2204.11172*, 2022. URL <https://arxiv.org/abs/2204.11172>.
- OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Brenden K. Petersen, Mikel Landajuela Larma, T. Nathan Mundhenk, Claudio P. Santiago, Soo K. Kim, and Joanne T. Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/1912.04871>.

- Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- Subham Sahoo, Christoph Lampert, and Georg Martius. Learning equations for extrapolation and control. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. URL <https://arxiv.org/abs/1806.07259>.
- Fangzheng Sun, Yang Liu, Jian-Xun Wang, and Hao Sun. Symbolic physics learner: Discovering governing equations via monte carlo tree search. In *International Conference on Learning Representations*, 2023.
- Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020.
- Nguyen Quang Uy, Nguyen Xuan Hoai, Michael O’Neill, Robert I. McKay, and Edgar Galván-López. Semantically-based crossover in genetic programming: application to real-valued symbolic regression. *Genetic Programming and Evolvable Machines*, 12(2):91–119, 2011.
- Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.
- David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.
- Keyulu Xu, Mozhi Zhang, Jingling Li, et al. How neural networks extrapolate: From feedforward to graph neural networks. In *ICLR 2021*, 2021.

Appendix A. Full Benchmark Case Definitions

A.1 Automated Market Makers

- **Constant product formula:** $x \cdot y = k$. Tests basic AMM invariant preservation.
- **Spot price from AMM:** $P = y/x$. Algebraic; should be trivially recovered.
- **Spot price from AMM reserve:** $P = \Delta y / \Delta x$. Marginal price from reserve changes.
- **AMM output amount:** $\Delta y = y - k / (x + \Delta x)$. Tests multi-step algebraic composition.
- **Price slippage percentage:** $\text{slip} = (P_{\text{exec}} - P_{\text{spot}}) / P_{\text{spot}}$.
- **Constant Product Price Impact:** quadratic impact formula.
- **LP share percentage:** $s = \Delta L / L_{\text{total}}$.

- **LP fee earnings:** $e = s \cdot F \cdot V$, where F is fee tier and V volume.
- **Impermanent loss percentage:** $IL = 2\sqrt{r}/(1+r) - 1$, where r is the price ratio at withdrawal vs. deposit.
- **Impermanent loss in constant product:** related quantity for the constant-product setting specifically.
- **Impermanent loss breakeven fee rate:** the fee rate F at which LP fees exactly offset impermanent loss.
- **Uniswap V3 virtual:** virtual liquidity quantity for concentrated positions.
- **Concentrated liquidity position width:** tick range formula.
- **Capital efficiency:** ratio of effective liquidity to deposited capital.

A.2 Lending Protocols

- **Loan-to-Value:** $LTV = D/C$.
- **Health factor:** $h = C_{\text{adj}}/D$.
- **Collateral ratio:** $CR = C/D$.
- **Reserve ratio:** $\rho = R/(R + L)$.
- **Utilization rate of DeFi:** $u = L/(L + R)$.
- **Borrow APY from utilization:** kinked interest rate model.
- **Supply APY from borrow:** $r_s = r_b \cdot u$.
- **Partial liquidation amount:** amount of collateral seized.
- **Liquidation Price Long and Liquidation Price Short:** prices at which a position is liquidated.
- **Leveraged position notional:** $N = C \cdot L$, where L is leverage.
- **Effective Leverage:** $L_{\text{eff}} = N/C$.
- **Maximum safe leverage:** function of maintenance margin rate.
- **Required collateral:** collateral needed for a given position.
- **Cross-margin available balance:** net balance accounting for all positions.
- **Multi-Collateral LTV:** weighted LTV across multiple collateral types.
- **Funding-Adjusted Liquidation Price (Long) and Funding-Adjusted Liquidation Price (Short):** extended formulas accounting for funding rates.

A.3 Derivatives Pricing

- **Black–Scholes Call Price:** $C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2)$, with $d_{1,2} = [\ln(S/K) + (r \pm \frac{1}{2}\sigma^2)T]/(\sigma\sqrt{T})$.
- **Black–Scholes Put Price:** $P = Ke^{-rT}\Phi(-d_2) - S\Phi(-d_1)$.
- **Delta:** $\Delta = \Phi(d_1)$ (call).
- **Gamma:** $\Gamma = \phi(d_1)/(S\sigma\sqrt{T})$.
- **Theta:** $\Theta = -S\phi(d_1)\sigma/(2\sqrt{T}) - rKe^{-rT}\Phi(d_2)$.
- **Vega:** $\mathcal{V} = S\phi(d_1)\sqrt{T}$.
- **Put-call parity:** $C - P = S - Ke^{-rT}$.
- **Forward price for derivative:** $F = Se^{rT}$.
- **Call option intrinsic:** $\max(S - K, 0)$.
- **Simple options moneyness:** $(S - K)/K$.
- **Convexity Adjustment:** bond convexity formula.

A.4 Risk and Portfolio Metrics

- **Value at Risk at 95 % and 99 %:** parametric VaR formulas.
- **Expected Shortfall at 95 % and 99 %:** conditional VaR formulas.
- **Multi-day Value at Risk:** $\text{VaR}_T = \text{VaR}_1 \cdot \sqrt{T}$.
- **ES scaling for multi-day:** analogous scaling for ES.
- **Incremental VaR:** marginal contribution of one asset to portfolio VaR.
- **Portfolio VaR for two correlated assets:** $\text{VaR}_P = \sqrt{w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\rho\sigma_1\sigma_2}$.
- **Correlated Portfolio VaR:** multivariate extension.
- **Component ES:** ES contribution decomposed by asset.
- **Portfolio Expected Shortfall for correlated assets:** the most complex risk case; requires integration over a multivariate Gaussian tail.
- **Annualised Portfolio tracking error:** $\text{TE} = \sigma_\epsilon\sqrt{252}$.
- **Portfolio Sharpe Ratio:** $(r_p - r_f)/\sigma_p$.
- **Information ratio:** $\text{IR} = \alpha/\text{TE}$.
- **Position margin ratio:** margin / position notional.

A.5 Staking and Protocol Mechanisms

- **Simple Staking APY:** $APY = r$.
- **Compounding Staking Returns:** $(1 + r/n)^{nT} - 1$.
- **Staking reward for fixed lock-up:** $R = P \cdot r \cdot T$.
- **Validator commission adjusted:** post-commission yield.
- **Slashing penalty:** fractional penalty applied to staked balance.
- **Protocol reserve accumulation:** cumulative reserve growth formula.
- **Funding rate cost:** $C = N \cdot f \cdot h$, where f is the funding rate and h is the holding period in funding intervals.
- **APY calculation:** annualised yield from periodic rates.

Appendix B. Benchmark Version History

Table 20: HypatiaX benchmark version history and key changes.

Version	Cases	Key Changes
v1.0	62	Initial benchmark; axis-aligned splits; no trust gating.
v2.0	71	PCA-directed splits introduced; trust gate added ($R^2 > 0.1$).
v3.0	74	Three hard cases added; trust gate raised to $R^2 > 0.5$; data leakage fixed; unified executor.

Appendix C. Runtime Reproducibility Audit: Detailed Analysis

The authoritative runtime table is Table 10 in §10.5 (main text), together with the LLM-routed-only view in Table 11. Both are the direct, unmodified output of `scripts/generate_tables.py` run against the five 74-task v3 seed files and the five 74-task PCA seed files in the repository, with no hand-editing of any number.

Two earlier speedup claims do not reproduce: a “73 % runtime reduction” ($3.7\times$ speedup), and a $1.73\times/1.64\times$ speedup. Neither reproduces from any raw file examined, under any seed or pooling; the source of the 73 % figure is not identified in any raw file audited.

All five seed files used here contain the full 74 tasks (§10.5). The pooled 5-seed / 370-task row in Table 10 is representative of those five files, with the provenance caveats given in §10.5, and the direction of the runtime claim (hybrid slower than NN, every seed) does not depend on the pooling.

Appendix D. Process and Credibility Notes

None of the items below is a claim made in the body of this paper; each bears on figures elsewhere in the reproducibility material. Full detail is in Supplementary Material A, Section 10 (“Process/Credibility Items”). The project’s CI dashboard from the July 9 run (Supplementary A §10.1) should not be cited: its upstream notebooks reported “not found” while its summary asserted “0 open issues”. The pipeline defect behind it (`build_report.py` accepting notebook file existence as a proxy for a completed run) is fixed as of commit `9d7a833f` (2026-07-29): the script now checks each notebook’s per-cell execution status and fails the build rather than publish a stale summary. The legacy `exp2` baseline-lock threshold defect ($R^2 \geq 0.9999$ used in place of the pipeline’s stated $R^2 \geq 0.999999$, which had inflated the legacy baseline from 74/90 to 81/90; Supplementary A §10.2) is resolved: 74/90 is confirmed against a fresh CI run and reproduced independently via a patched Gate C schema check (§10.4), so 74/90 should be cited, not 81/90. The $1.73\times$ runtime-speedup figure has been audited and does not reproduce (see §10.5 and the abstract). The remaining resolved sub-items (the `ablation_paired.json` 21/30 vs. 20/30 discrepancy, §10.6, and the Gate C `source_files` manifest, §10.5) are described in Supplementary A and in the Post-Submission Report.